

A SAMPLING-BASED PLANNING METHOD FOR DYNAMIC  
OPTIMIZATION PROBLEMS

A THESIS SUBMITTED TO  
THE GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES  
OF  
MIDDLE EAST TECHNICAL UNIVERSITY

BY

EGE UĞUR AGUŞ

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS  
FOR  
THE DEGREE OF MASTER OF SCIENCE  
IN  
MECHANICAL ENGINEERING

SEPTEMBER 2023



Approval of the thesis:

**A SAMPLING-BASED PLANNING METHOD FOR DYNAMIC  
OPTIMIZATION PROBLEMS**

submitted by **EGE UĞUR AGUŞ** in partial fulfillment of the requirements for the degree of **Master of Science in Mechanical Engineering Department, Middle East Technical University** by,

Prof. Dr. Halil Kalıpçılar  
Dean, Graduate School of **Natural and Applied Sciences**

\_\_\_\_\_

Prof. Dr. Mehmet Ali Sahir Arıkan  
Head of Department, **Mechanical Engineering**

\_\_\_\_\_

Assoc. Prof. Dr. Ali Emre Turgut  
Supervisor, **Mechanical Engineering, METU**

\_\_\_\_\_

Assoc. Prof. Dr. Ahmet Buğra Koku  
Co-supervisor, **Mechanical Engineering, METU**

\_\_\_\_\_

**Examining Committee Members:**

Assoc. Prof. Dr. Ahmet Buğra Koku  
Mechanical Engineering, METU

\_\_\_\_\_

Assoc. Prof. Dr. Ali Emre Turgut  
Mechanical Engineering, METU

\_\_\_\_\_

Prof. Dr. Gökçe Nur Yılmaz  
Computer Engineering, TED University

\_\_\_\_\_

Assist. Prof. Dr. Kutluk Bilge Arıkan  
Mechanical Engineering, TED University

\_\_\_\_\_

Assoc. Prof. Dr. Ender Yıldırım  
Mechanical Engineering, METU

\_\_\_\_\_

Date:07.09.2023

**I hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct, I have fully cited and referenced all material and results that are not original to this work.**

Name, Surname: Ege Uğur Ağuş

Signature :

## **ABSTRACT**

### **A SAMPLING-BASED PLANNING METHOD FOR DYNAMIC OPTIMIZATION PROBLEMS**

Aguş, Ege Uğur

M.S., Department of Mechanical Engineering

Supervisor: Assoc. Prof. Dr. Ali Emre Turgut

Co-Supervisor: Assoc. Prof. Dr. Ahmet Buğra Koku

September 2023, 89 pages

This thesis investigates a new sampling-based planning method for efficiently solving complex dynamic optimization problems that other sampling-based planning methods do not provide optimal solutions. Proposed sampling-based planning method achieves this by using a new search method, studied in this thesis, possible optimal paths search, POPS. Possible optimal paths search is a search method that finds all paths that are optimal for at least one set of weights of the edges or nodes of a graph. Proposed sampling-based planning method uses the basics of the POPS to find alternative paths from an initial state to goal state.

The proposed sampling-based planning method is designed to operate in real-time. It provides optimality by finding the paths from initial state to goal state with the minimum number of state changes. Outputs of the proposed method are the inputs for any discrete optimization methods which found in the literature. This way complex dynamic optimization problems can be solved efficiently.

Keywords: Sampling-based planning, Discrete optimization, Planning algorithms,  
Dynamic optimization, Search methods

## ÖZ

### **DİNAMİK OPTİMİZASYON PROBLEMLERİ İÇİN ÖRNEK TABANLI PLANLAMA YÖNTEMİ**

Aguş, Ege Uğur

Yüksek Lisans, Makina Mühendisliği Bölümü

Tez Yöneticisi: Doç. Dr. Ali Emre Turgut

Ortak Tez Yöneticisi: Doç. Dr. Ahmet Buğra Koku

Eylül 2023 , 89 sayfa

Bu tezde, diğer örnekleme tabanlı planlama yöntemlerinin optimal çözümler sağlamadığı karmaşık dinamik optimizasyon problemlerini verimli bir şekilde çözmek için yeni bir örnekleme tabanlı bir planlama yöntemi çalışılmıştır. Önerilen örnekleme tabanlı planlama yöntemi bunu, yine bu tezde incelenen yeni bir arama yöntemi olan olası optimal yollar aramasını, POPS, kullanarak başarır. Olası optimal yollar araması, bir grafiğin kenarlarının veya düğümlerinin en az bir ağırlık seti için optimal olan yolların, tümünü bulan bir arama yöntemidir. Önerilen örnekleme tabanlı planlama yöntemi, başlangıç durumundan hedef duruma alternatif yollar bulmak için POPS'un temellerini kullanır.

Önerilen örnekleme tabanlı planlama yöntemi, gerçek zamanlı çalışacak şekilde tasarlanmıştır. Minimum sayıda durum değişikliği ile başlangıç durumundan hedef duruma giden yolları bularak optimallliği sağlar. Önerilen yöntemin çıktıları, literatürde bulunan herhangi bir ayrık optimizasyon yöntemi için girdi niteliğindedir. Bu şekilde karmaşık dinamik optimizasyon problemleri verimli bir şekilde çözülebilir.

Anahtar Kelimeler: Örnek tabanlı planlama, Ayrık optimizasyon, Planlama algoritmaları, Dinamik optimizasyon, Arama metotları

To my better half, who inspires and transforms me into a better version of myself.

## ACKNOWLEDGMENTS

I would like to express my most profound appreciation to my supervisor, Assoc. Prof. Dr. Ali Emre Turgut, for his invaluable assistance, leadership, and coaching in my academic and professional life. His knowledge, passion, and understanding were some of the critical components in my success in overcoming the difficulties I encountered throughout the thesis. It was truly a privilege to be able to study under his supervision.

I am incredibly grateful for the invaluable advice, assistance, and guidance provided by my co-supervisor, Assoc. Prof. Dr. A. Buğra Koku. He played a significant role in my personal and academic life, helping me rediscover my passion for engineering. His innovative ideas were a crucial factor in completing this thesis, and I feel incredibly fortunate to have had the opportunity to work with him.

I am immensely grateful to my wife for her love and constant support throughout my thesis journey. Her unwavering belief in me, encouragement, and understanding have been my guiding light, enabling me to achieve this accomplishment. She has been my constant source of inspiration, and I cannot thank her enough for sharing this accomplishment with me.

I am grateful to my parents and brother for their unwavering love, motivation, and constant backing in all my academic endeavors. Their unwavering faith in me has been the cornerstone of my achievements.

I want to thank the friends who are always there. Thanks to their support, I have always been able to stay motivated.

My thesis would not have come to completion without the unwavering support and encouragement of the people I mentioned. I am grateful and humbled by their profound influence on my success.

## TABLE OF CONTENTS

|                                                 |       |
|-------------------------------------------------|-------|
| ABSTRACT . . . . .                              | v     |
| ÖZ . . . . .                                    | vii   |
| ACKNOWLEDGMENTS . . . . .                       | x     |
| TABLE OF CONTENTS . . . . .                     | xi    |
| LIST OF FIGURES . . . . .                       | xv    |
| LIST OF ABBREVIATIONS . . . . .                 | xviii |
| CHAPTERS                                        |       |
| 1 INTRODUCTION . . . . .                        | 1     |
| 1.1 Motivation and Problem Definition . . . . . | 1     |
| 1.2 Proposed Methods and Models . . . . .       | 4     |
| 1.3 Contributions and Novelties . . . . .       | 5     |
| 1.4 The Outline of the Thesis . . . . .         | 5     |
| 2 LITERATURE SURVEY . . . . .                   | 7     |
| 2.1 Introduction . . . . .                      | 7     |
| 2.2 Planning Algorithms . . . . .               | 7     |
| 2.2.1 Search Methods . . . . .                  | 8     |
| 2.2.1.1 Uninformed Search . . . . .             | 9     |
| Breadth First Search . . . . .                  | 9     |

|                                               |    |
|-----------------------------------------------|----|
| Depth First Search . . . . .                  | 10 |
| 2.2.1.2 Informed Search . . . . .             | 10 |
| Dijkstra's Algorithm . . . . .                | 10 |
| A-star Algorithm . . . . .                    | 10 |
| Best First Search . . . . .                   | 11 |
| 2.2.2 Combinatorial Planning . . . . .        | 12 |
| Roadmap . . . . .                             | 12 |
| 2.2.2.1 Vertical Cell Decomposition . . . . . | 12 |
| 2.2.2.2 Shortest Path Roadmap . . . . .       | 13 |
| 2.2.3 Sampling-Based Planning . . . . .       | 15 |
| 2.2.3.1 Sampling Methods . . . . .            | 15 |
| Random-Uniform Sampling . . . . .             | 15 |
| Low-Dispersion Sampling . . . . .             | 16 |
| Low-Discrepancy Sampling . . . . .            | 16 |
| Boundary Sampling . . . . .                   | 16 |
| Bridge Test Sampling . . . . .                | 16 |
| Medial-axis Sampling . . . . .                | 16 |
| 2.2.3.2 Single-Query Models . . . . .         | 17 |
| Rapidly exploring random tree, RRT . . . . .  | 18 |
| RRT* . . . . .                                | 18 |
| Fast Marching Tree, FMT* . . . . .            | 18 |
| Informed RRT* . . . . .                       | 18 |
| Batch Informed Trees, BIT* . . . . .          | 19 |

|         |                                                  |    |
|---------|--------------------------------------------------|----|
| 2.2.3.3 | Multi-Query Models . . . . .                     | 19 |
|         | Probabilistic Roadmap, PRM . . . . .             | 19 |
|         | Probabilistic Roadmap Star, PRM* . . . . .       | 20 |
|         | Visibility Roadmap, VRM . . . . .                | 20 |
| 2.2.4   | Discrete Optimization . . . . .                  | 21 |
|         | Branch and Bound . . . . .                       | 21 |
|         | Constrained Programming . . . . .                | 21 |
|         | Local Search . . . . .                           | 21 |
|         | Linear Programming . . . . .                     | 22 |
|         | Genetic Algorithms and Meta-heuristics . . . . . | 22 |
|         | Large Neighborhood Search, LNS . . . . .         | 22 |
| 3       | METHODOLOGY . . . . .                            | 25 |
| 3.1     | Building Paths . . . . .                         | 25 |
| 3.2     | POPS: Possible Optimal Paths Search . . . . .    | 29 |
| 3.3     | Building Roadmap . . . . .                       | 32 |
| 3.4     | Algorithm Design . . . . .                       | 35 |
| 3.5     | Improving Algorithm's Performance . . . . .      | 37 |
| 3.6     | Building Regions . . . . .                       | 50 |
| 3.7     | Finding Solution to a Goal Node . . . . .        | 55 |
| 3.8     | Region Orders . . . . .                          | 56 |
| 4       | FORMULATION OF THE CASE STUDY . . . . .          | 59 |
| 4.1     | Formulation . . . . .                            | 59 |
|         | Constrain 1 . . . . .                            | 60 |

|         |                                                                     |    |
|---------|---------------------------------------------------------------------|----|
| 4.1.1   | Assumptions . . . . .                                               | 60 |
| 4.2     | Applying Discrete Optimization to Find Optimal Order of Targets . . | 60 |
| 5       | SIMULATION RESULTS AND DISCUSSION . . . . .                         | 63 |
| 5.1     | Simulation Results . . . . .                                        | 63 |
| 5.1.1   | Results for Map 1 . . . . .                                         | 63 |
| 5.1.1.1 | Map 1, Scenario 1 . . . . .                                         | 63 |
| 5.1.1.2 | Map 1, Scenario 2 . . . . .                                         | 64 |
| 5.1.2   | Results for Map 2 . . . . .                                         | 65 |
| 5.2     | Discussion . . . . .                                                | 66 |
| 6       | CONCLUSION . . . . .                                                | 77 |
|         | REFERENCES . . . . .                                                | 81 |
|         | APPENDICES                                                          |    |
| A       | EVALUATIONS OF THE METHODS IN THE LITERATURE SURVEY .               | 83 |
| A.1     | Method Evaluation . . . . .                                         | 83 |
| A.1.1   | Planning Algorithms' Evaluation . . . . .                           | 83 |
| A.1.2   | Discrete Optimization Algorithms' Evaluation . . . . .              | 86 |

## LIST OF FIGURES

### FIGURES

|             |                                                                                                            |    |
|-------------|------------------------------------------------------------------------------------------------------------|----|
| Figure 2.1  | Uninformed Search Tree . . . . .                                                                           | 9  |
| Figure 2.2  | Informed Search Tree . . . . .                                                                             | 11 |
| Figure 2.3  | Vertical Cell Decomposition Method . . . . .                                                               | 13 |
| Figure 2.4  | Shortest Path Roadmap Method . . . . .                                                                     | 14 |
| Figure 3.1  | An example roadmap built-in $C$ . . . . .                                                                  | 26 |
| Figure 3.2  | BFS Search over Figure 3.1 . . . . .                                                                       | 27 |
| Figure 3.3  | BFS Search without marking nodes as visited over figure 3.1,<br>redundant paths colored with red . . . . . | 28 |
| Figure 3.4  | BFS Search with only <b>sub-optimal</b> paths over figure 3.1 . . . . .                                    | 30 |
| Figure 3.5  | <b>POPS</b> algorithm . . . . .                                                                            | 31 |
| Figure 3.6  | An example graph to show <b>POPS</b> . . . . .                                                             | 32 |
| Figure 3.7  | BFS Search with only <b>proper</b> paths over figure 3.1, complete<br>paths colored with blue . . . . .    | 34 |
| Figure 3.8  | The roadmap built with complete paths in $C$ . . . . .                                                     | 35 |
| Figure 3.9  | Algorithm 1, Find Complete Paths . . . . .                                                                 | 36 |
| Figure 3.10 | Algorithm 2, Find Complete Paths with <b>seen nodes vectors</b> . . . . .                                  | 38 |
| Figure 3.11 | Example space densely sampled with 3000 nodes . . . . .                                                    | 40 |

|             |                                                                                                                                |    |
|-------------|--------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 3.12 | Illustration of excessive number of paths in 3.11 . . . . .                                                                    | 41 |
| Figure 3.13 | Illustration of extremely close sample nodes in $C_{free}$ . . . . .                                                           | 42 |
| Figure 3.14 | BFS Search with only <b>proper</b> paths of figure 3.13, new paths<br>over 3.7 shown with orange . . . . .                     | 42 |
| Figure 3.15 | Alternative scenario for of extremely close sample nodes in $C_{free}$                                                         | 43 |
| Figure 3.16 | BFS Search with only <b>proper</b> paths of figure 3.15 . . . . .                                                              | 43 |
| Figure 3.17 | Alternative scenario for of extremely close sample nodes in $C_{free}$ ,<br>with <b>dominated nodes</b> shown as red . . . . . | 44 |
| Figure 3.18 | BFS Search with only <b>proper</b> paths of figure 3.17 . . . . .                                                              | 44 |
| Figure 3.19 | Find Complete Paths with <b>candidates common vectors</b> and<br><b>dominated/dominating nodes</b> . . . . .                   | 45 |
| Figure 3.19 | Find Complete Paths with <b>candidates common vectors</b> and<br><b>dominated/dominating nodes</b> (cont.) . . . . .           | 46 |
| Figure 3.20 | A densely sampled, obstacle-rich $C_{space}$ . . . . .                                                                         | 47 |
| Figure 3.21 | Main paths over figure 3.20 . . . . .                                                                                          | 48 |
| Figure 3.22 | Adjacency matrix of figure 3.17 . . . . .                                                                                      | 50 |
| Figure 3.23 | Roadmap nodes, 198 nodes, over 3.11 . . . . .                                                                                  | 51 |
| Figure 3.24 | Complete paths, 1132 paths, over figure 3.11 . . . . .                                                                         | 52 |
| Figure 3.25 | Extra found nodes and paths by the algorithm . . . . .                                                                         | 53 |
| Figure 3.26 | Roadmap nodes, 127 nodes, over 3.11 with brotherhood . . . . .                                                                 | 54 |
| Figure 3.27 | Complete paths, 580 paths, over figure 3.11 with brotherhood . . . . .                                                         | 55 |
| Figure 3.28 | Regions, 69 region, over figure 3.11 . . . . .                                                                                 | 56 |
| Figure 3.29 | Different goals, big red nodes, and their solutions . . . . .                                                                  | 57 |

|             |                                                                                                                  |    |
|-------------|------------------------------------------------------------------------------------------------------------------|----|
| Figure 5.1  | Map 1 and created regions . . . . .                                                                              | 64 |
| Figure 5.2  | Map 1, Scenario 1 . . . . .                                                                                      | 65 |
| Figure 5.3  | Different goals, big red nodes, and their possible solutions for<br>map 1, scenario 1 with two drones . . . . .  | 69 |
| Figure 5.4  | Different goals, big red nodes, and their possible solutions for<br>map 1, scenario 1 with five drones . . . . . | 70 |
| Figure 5.5  | Solution of the case study for map 1 for scenario 1 with two drones                                              | 71 |
| Figure 5.6  | Map 1, Scenario 2 . . . . .                                                                                      | 72 |
| Figure 5.7  | Solution of the case study for map 1, scenario 2 with two drones                                                 | 72 |
| Figure 5.8  | Map 2 and created regions . . . . .                                                                              | 73 |
| Figure 5.9  | Map 2, scenario . . . . .                                                                                        | 73 |
| Figure 5.10 | Different goals, big red nodes, and their possible solutions for<br>map 2's scenario with two drones . . . . .   | 74 |
| Figure 5.11 | Different goals, big red nodes, and their possible solutions for<br>map 2's scenario with three drones . . . . . | 75 |
| Figure 5.12 | Solution of the case study for map 2's scenario with three drones                                                | 76 |
| Figure A.1  | Example Scenario . . . . .                                                                                       | 84 |
| Figure A.2  | Dense Sampled State-Space with 3000 nodes . . . . .                                                              | 85 |
| Figure A.3  | Example scenario where multiple paths share the same node . . .                                                  | 86 |
| Figure A.4  | Application of RRT . . . . .                                                                                     | 87 |
| Figure A.5  | Application of RRT* . . . . .                                                                                    | 88 |
| Figure A.6  | Application of RRT and RRT* with a narrower passage . . . . .                                                    | 89 |
| Figure A.7  | 3 different Visibility Roadmaps over the state-space . . . . .                                                   | 89 |

## **LIST OF ABBREVIATIONS**

|      |                               |
|------|-------------------------------|
| 2D   | 2 Dimensional                 |
| 3D   | 3 Dimensional                 |
| BFS  | Breadth First Search          |
| DFS  | Depth First Search            |
| DOPs | Dynamic Optimization Problems |
| LaWS | Laser Weapon System           |
| POPS | Possible Optimal Paths Search |
| UAV  | Unmanned Aerial Vehicle       |

## CHAPTER 1

### INTRODUCTION

#### 1.1 Motivation and Problem Definition

In many optimization problems, especially in real-world scenarios such as networks, scheduling, and robotics, elements are not static but rather dynamic. These dynamic elements can generally be changing environmental conditions, uncertain inputs, space- or time-varying constraints, and objectives. These kinds of problems are called dynamic optimization problems, or DOPs. Due to their highly complex nature and dynamics, traditional methods might struggle to handle DOPs' complexities efficiently. The most general methods to solve DOPs are evolutionary optimization techniques, swarm intelligence, and other meta-heuristic techniques, [1], [2]. DOPs can be categorized broadly as combinatorial, continuous, multi-objective, single-objective, unconstrained, and constrained. However, no definitively accepted definition of DOPs is used by everyone in the literature, [3]. This thesis studies DOPs with dynamic objective functions that vary with the current state of the world and dynamic goals, where new goals are defined over time.

Sampling-based planning is an extensively studied approach in robotics, developed to solve high-dimensional or complex problems that combinatorial planning methods cannot solve. The primary approach of these methods is to avoid the construction of the obstacle space since it is highly costly and impractical in many situations. Sampling-based planning methods showed superior performance in the late 1990s [4], [5], and became a hot topic among researchers, especially in motion planning. They are mainly classified as multi-query and single-query. Multi-query sampling-based planners employ a preprocessing step to establish connectivity relationships.

Typically, they construct a roadmap within the solution space and utilize it to facilitate transformations from any initial state to any goal state. This approach effectively utilizes the connectivity information inherent in the solution space. On the other hand, single-query sampling-based planners adopt a specific search methodology, often involving the construction of tree structures within the solution space for a given initial and goal state. Unlike multi-query planners, single-query planners do not employ a preprocessing phase.

This thesis aims to develop a sampling-based approach for solving dynamic optimization problems characterized by varying objective functions and goals. The central concept revolves around leveraging the efficient exploration capabilities of sampling-based planning to address the complex decision-making processes inherent in dynamic optimization problems. Sampling-based methods are generic, allowing for their adaptation to various other problems. Furthermore, their effectiveness within high-dimensional or complex environments indicates the potential success of sampling-based planning methods in solving dynamic optimization problems.

In order to successfully develop and test the proposed sampling-based method in this thesis, we chose a sample case study. The selected case study consists of unmanned aerial vehicles, UAVs, and a laser weapon system, LaWS. UAVs generally offer high mobility and aerial perspective while operating without human intervention. UAVs can significantly reduce the time and resources required for activities such as surveys, inspections, and deliveries with the ability to access remote or hazardous locations. UAVs equipped with sensors enable advanced data collection and analysis. Moreover, utilizing UAVs in tasks that involve risk to human life can improve safety conditions.

Alongside their advantages, UAVs also pose specific threats that must be carefully addressed. The main threats of UAVs are privacy concerns, cybersecurity risks, and criminal activities. Unauthorized or unethical use of data collected by UAVs can compromise personal privacy. Hackers can gain unauthorized access to the vehicles' control systems, compromising their operation or stealing sensitive data. Such attacks threaten public safety, national security, and personal privacy. Lastly, UAVs can be employed for illegal activities. They can be weaponized, used for unauthorized surveillance, or deployed to disrupt critical infrastructure.

Researchers and defense organizations have explored various technological approaches to counter UAVs and address potential threats. One of the technological research areas aimed at taking down unmanned vehicles is laser weapon systems. Laser Weapon Systems, LaWSs, are directed energy weapons that utilize laser technology to engage and disable targets. They are directed energy weapons that focus and emit a concentrated beam of coherent light energy, typically in the form of lasers, to damage or destroy a target. The LaWSs generate a high-energy laser beam and direct it towards a target. When the laser beam interacts with the target, it rapidly heats and vaporizes the material, causing localized damage. The energy from the laser beam can be absorbed by the target's surface, leading to melting, burning, or structural weakening, depending on the intensity and duration of the laser exposure. LaWS provides high speed, accuracy, and reduced collateral damage. LaWS have been primarily developed for defense and security applications, including ship-based defense against UAVs. They are also being explored for potential use in missile defense, precision targeting of enemy assets, and protection of military installations.

LaWSs have certain limitations and disadvantages that should be considered. Some main disadvantages of LaWSs are atmospheric conditions, power requirements, thermal management, range limitations, countermeasures, and complexity. Atmospheric conditions such as fog, smoke, dust, or precipitation can affect the laser beams by scattering or absorbing the laser energy, reducing its effectiveness and range, particularly for long-range engagements. LaWSs require significant power to generate and sustain the high-energy laser beams. This need necessitates a robust and reliable power source. High-energy laser beams generate substantial heat during operation, potentially damaging or degrading the laser system. Effective thermal management systems are required to dissipate this heat and prevent overheating, ensuring the longevity and reliability of the laser weapon system. Adversaries may develop countermeasures, such as reflective surfaces or coatings, to mitigate the effectiveness of laser weapon systems.

The selected case study will be defined as exploring the feasibility and optimization of delivering laser beams to sequential targets using drones that can efficiently transmit laser beams. By leveraging the flexibility and mobility of UAVs, we aim to develop a system that can efficiently transmit laser beams from a central laser source to a

series of predetermined targets. This approach can neutralize many disadvantages of LaWSs and increase their operational capability. This approach allows LaWSs to hit targets that are not in the line of sight of the LaWSs and increase the range of application by avoiding dust in the atmosphere or obstacles in the area. This capability allows LaWSs to defend an area with obstacles by keeping the LaWSs fixed due to its infrastructure requirements.

In the selected case study, drones must be placed strategically to eliminate targets. However, optimally ordering targets is a challenging task. Since the environment is obstacle-rich, there are many possible different placements of the drones to eliminate the targets. Moreover, these different sets of placements to eliminate targets have varying costs with the current positions of the drones. Therefore, the sequence and orientations for eliminating the targets remain unclear. Moreover, a new target can be defined at any point in the problem, and the method should rapidly adapt to the new target. In this case study, the objective function and the goal are dynamic.

## **1.2 Proposed Methods and Models**

The proposed method builds a roadmap that contains possible alternative paths for the discrete optimization algorithms. A roadmap is built by processing the operation field, and it is designed to be minimal to decrease the search cost at the discrete optimization algorithms. Moreover, the roadmap is designed to be optimal for any state of the world and any goal defined in the world. Thus, the optimal solution is encompassed within the roadmap for any form of the dynamic objective function and goal configuration. Optimality of the roadmap is achieved by a new search approach, POPS, which will be introduced later on in this study. Once a roadmap is constructed, any discrete optimization technique can be used to solve dynamic optimization problems to achieve optimal sorting of targets based on dynamic objective function. In the selected case study, the objective function is selected based on minimizing the mission time to eliminate all targets defined in the world.

### **1.3 Contributions and Novelties**

Our contributions are as follows:

- A new search method for a single source all possible optimal paths to nodes in a graph.
- A new hybrid Roadmap-Tree method that can be used in planning problems. Moreover, the method includes all alternative solutions for dynamic optimization problems.
- A novel system that increases operational capabilities of the Laser Weapon Systems.

### **1.4 The Outline of the Thesis**

The thesis continues with a literature survey covering search methods, planning algorithms, and discrete optimization methods in Chapter 2. Chapter 3 is devoted to formulating the problem and evaluating the algorithms mentioned in Chapter 2. Chapter 4 introduces a novel search method and proposed sampling-based planning method that uses the introduced search method as its basics and application of the proposed method into the case study. Chapter 5 is committed to the simulation results. Finally, in Chapter 5, the conclusion of this thesis is presented.



## CHAPTER 2

### LITERATURE SURVEY

#### 2.1 Introduction

This chapter presents a literature survey about planning algorithms and discrete optimization methods. First, studies about planning algorithms and then studies about discrete optimization methods are shared in chronological order.

#### 2.2 Planning Algorithms

A *planning problem* is a fundamental concept that involves determining a sequence of actions or steps to achieve a specific goal state from a given initial state, while adhering to a set of constraints or optimizing particular objectives. Each distinct situation in the world is called a state, denoted by  $x$ , and the set of all possible states is called a state space,  $X$ . The state space captures the relevant information about the system's current condition and encapsulates the entire planning problem.

States may be transformed through the application of actions. Each action,  $u$ , when applied from the current state,  $x$ , produces a new state,  $x'$ , as specified by a state transition function,  $f$ . Transformation can be expressed as,

$$\mathbf{x}' = \mathbf{f}(\mathbf{x}, \mathbf{u}) \quad (2.1)$$

Then, the planning problem model can be stated as [6],

- Let's define  $X$  as a nonempty state space.

- There is a finite action space  $U(x)$ , for each state  $x \in X$ .
- Define a state transition function  $f$  that produces a state  $f(x, u) = x', x' \in X$  for every  $x \in X$  and  $u \in U(x)$ .
- Define an initial state  $x_i \in X$ .
- Define an goal state  $x_g \in X$ .

The solution is found by finding a set of actions that can transform  $x_i$  to  $x_g$ . A planner can search for a feasible or optimal solution. A feasible solution requires finding a set of actions that transform  $x_i$  to  $x_g$  without any other criteria. An optimal solution requires finding a set of actions that transform  $x_i$  to  $x_g$  subject to an objective function that any of the other feasible sets can improve its value. Regardless of the problem type, a *search method* should be applied to find a solution. However, many planning problems are defined in *continuous spaces*, and transformation to the *discrete space* is favorable since searching continuous spaces containing uncountably infinite states is generally more challenging than searching discrete spaces containing finite states. This transformation is generally obtained by two methods, *combinatorial planning* and *sampling-based planning*.

### 2.2.1 Search Methods

Search algorithms are fundamental techniques to explore possible paths from an initial state to a goal state. It involves systematically generating successor states from the current state and evaluating their feasibility and desirability based on specific criteria, such as optimally, cost, or heuristic estimates. The search process continues until a goal state is reached or a termination condition is met.

Search algorithms can be broadly categorized into uninformed and informed, depending on whether they use some information, named heuristics, to guide the search.

### 2.2.1.1 Uninformed Search

Uninformed search algorithms explore the search space without additional information about the problem. Uninformed search algorithms typically rely on basic strategies, such as breadth-first search, BFS, and depth-first search, DFS, to traverse the state space systematically.

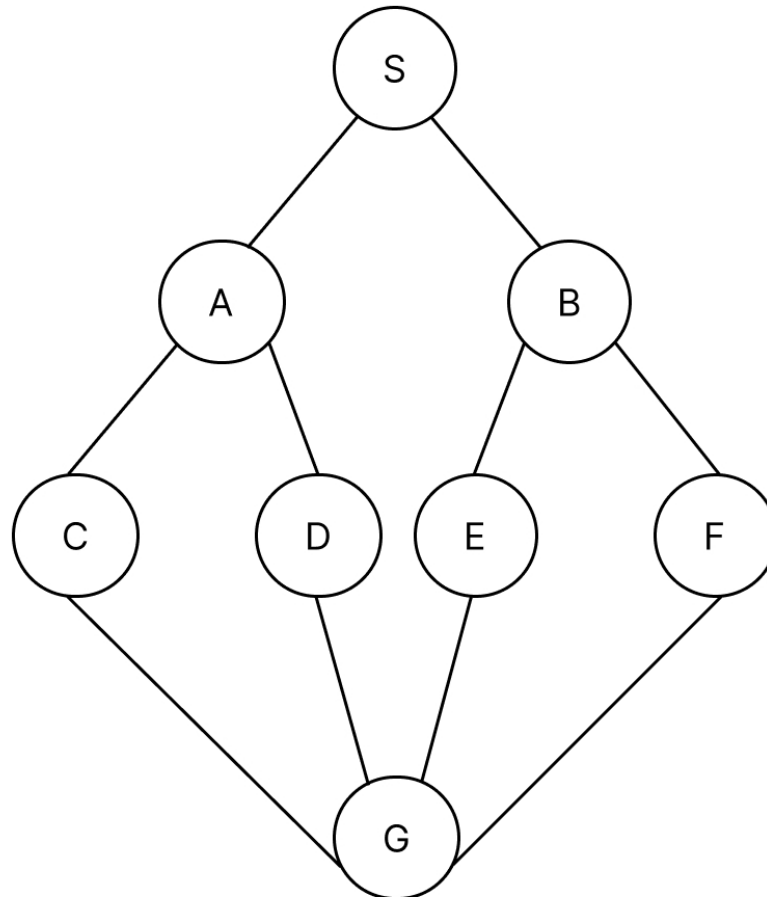


Figure 2.1: Uninformed Search Tree

**Breadth First Search** BFS is an uninformed search algorithm that explores all possible states at a given depth level before moving on to the next level, [7]. It guarantees finding the optimal path to the goal state in a tree or graph search space. However, its memory requirements can be substantial, making it less efficient for large state spaces. If we assume BFS prioritizes the left children in the binary tree shown in figure 2.1

search order from the node  $S$ , the BFS becomes  $S - A - B - C - D - E - F - G$ .

**Depth First Search** DFS is an uninformed search algorithm that explores the search space by continuously exploring a path until it reaches a leaf node or encounters a dead-end. DFS then backtracks and explores other branches of the search tree. DFS has low memory requirements compared to BFS but may not guarantee finding the optimal path to the goal state. If we assume DFS prioritizes the left children in the binary tree shown in figure 2.1 search order from the node  $S$ , the DFS becomes  $S - A - C - G - D - B - E - F$ .

### 2.2.1.2 Informed Search

Informed search algorithms, also known as heuristic search algorithms, incorporate domain-specific knowledge to guide the search process efficiently towards the goal state. They use heuristics to estimate states' desirability and prioritize exploring more promising paths. Figure 2.2 is designed as  $S$  is the source node and  $G$  is the goal node. Unlike the figure 2.1, figure 2.2 has heuristic values, which try to guess actual cost-to-arrive values from specific nodes.

**Dijkstra's Algorithm** Dijkstra's algorithm is a method to find single-source shortest paths to all nodes in a graph, [8]. The algorithm initializes by setting the cost-to-arrive value to infinity for every node except the source node. Then, the algorithm calculates the tentative distance from the start node to the neighbor through the current node. This distance is the sum of the distance from the start node to the current node and the weight of the edge between the current node and the neighbor. If the tentative distance is smaller than the previous distance recorded in the memory, update the memory with the new smaller distance. Once all nodes have been visited, the algorithm finds minimum values for every other node from the source node.

**A-star Algorithm** A-star Algorithm is an extension of Dijkstra's Algorithm that tries to reduce the total number of states explored by incorporating a heuristic estimate of the cost-to-arrive value to the goal from a given state, [9]. The algorithm explores

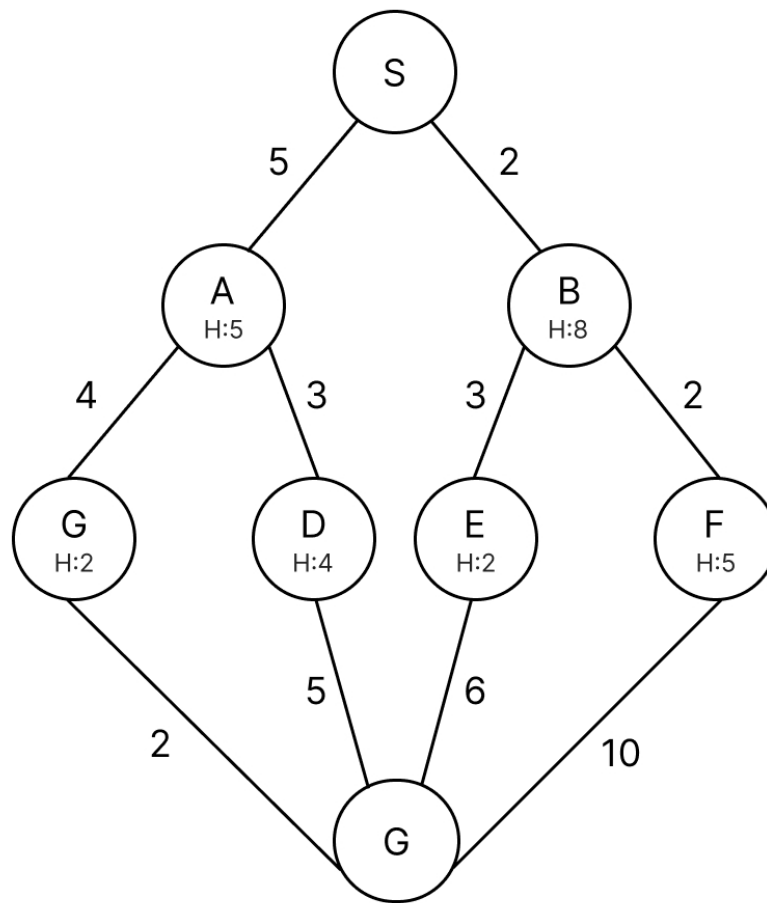


Figure 2.2: Informed Search Tree

the states with the lowest estimated total cost, the sum of the actual cost, and the heuristic cost. A\* guarantees finding an optimal solution if the heuristic is admissible (never overestimates the actual cost) and consistent.

**Best First Search** Best First Search is a method that sorts the neighbors only according to a heuristic estimate of the cost-to-arrive value. This approach generally leads to non-optimal solutions since the algorithm always picks the best-looking neighbor. Although optimal solutions are not found, far fewer vertices are explored in many cases, which results in much faster running times. However, the best first search's worst-case performance is worse than the A-star search.

### 2.2.2 Combinatorial Planning

Combinatorial planning methods find connections in the continuous space without using approximations. Moreover, combinatorial planning algorithms are complete, which means the method can find a solution if a solution exists. If not, the method can detect that there is no solution to that problem. They might be optimal. The optimality of the combinatorial planning methods depends on the specific method. All combinatorial planning methods build a roadmap to solve the planning problems.

**Roadmap** Roadmaps reflect the connectivity of free space by building a graph from the states contained in the space. They are the union of states that can transform each other by state transition function, such that for all initial,  $x_i$ , and goal,  $x_g$ , states in state space,  $X$ , can be transformed by using states in the roadmap as intermediate states.

Unfortunately, many combinatorial planning methods are inefficient, impractical, and difficult to implement. However, some might outperform sampling-based methods in specific 2D problems, and they provide theoretical upper bounds on the time needed to solve planning problems. In this survey, *Vertical Cell Decomposition* and *Shortest Path Roadmap* are presented as combinatorial planning methods.

#### 2.2.2.1 Vertical Cell Decomposition

In order to define the Vertical Cell Decomposition, let  $V$  denote the set of vertices used to define obstacles in the space. At every  $v \in V$ , extend rays upward and downward through free space until a ray reaches an obstacle or the end of the free space, [10]. Vertical decomposition is obtained by partitioning the free space according to these rays. Figure 2.3 shows the vertical cell decomposition process.

Figure 2.3a shows the space chosen to apply VCD. Figure 2.3b shows the decomposed space by the rays. After the decomposition, a roadmap is built from the cell centroids. In figure 2.3c, the built roadmap is shown, and in figure 2.3d an example query is shown on the right. When an initial state  $x_i$  and goal state  $x_g$  are defined, both are connected to their cells' centroid. After that point, a search method is applied to find

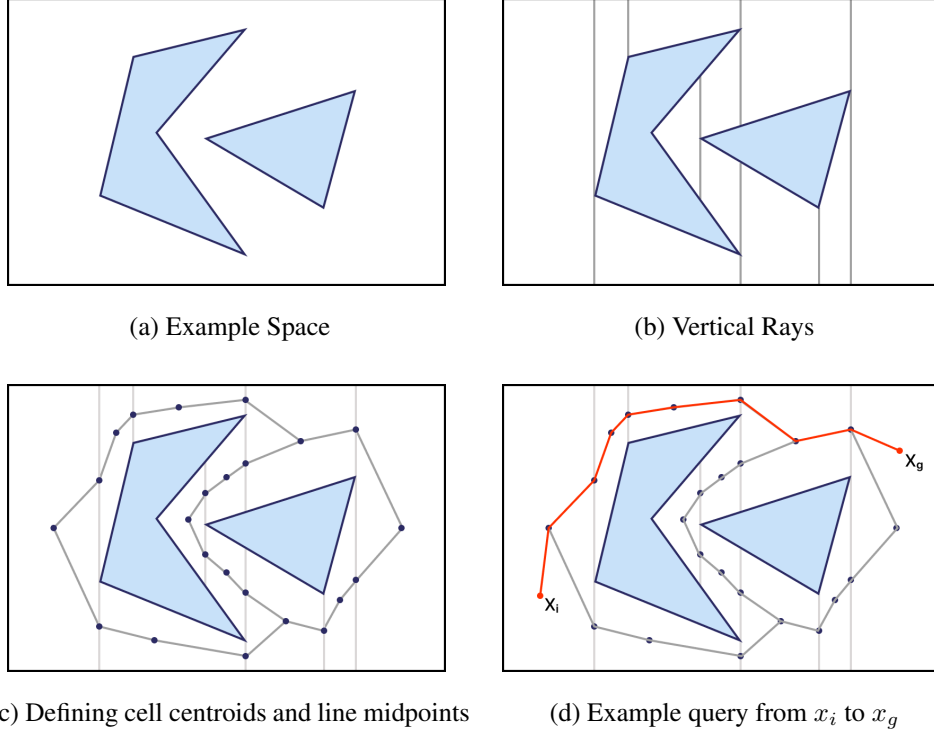


Figure 2.3: Vertical Cell Decomposition Method

the path to transform the  $x_i$  to the  $x_g$ .

This method can perform well in 2D space filled with a few obstacles. However, in 3D, defining the vertices of the obstacles and calculating the decomposition takes much work.

### 2.2.2.2 Shortest Path Roadmap

Shortest Path Roadmap provides the shortest transformations in the free space from any  $x_i$  to  $x_g$ . Let  $V$  denote the set of vertices used to define obstacles in the space. At every  $v \in V$ , define the actions that transforms  $v$  to  $v_n$  for every  $v_n \in V - \{v\}$ . If the action and its extension are feasible, add  $v$  and  $v_n$  and connect them with the specified action. In the end, the shortest path roadmap is constructed. After the roadmap is constructed,  $x_i$  and  $x_g$  are connected to a vertex if a feasible action transforms  $x_i$  or  $x_g$  to that vertex. Then, a search method is applied to find the path to transform the  $x_i$  to the  $x_g$ .

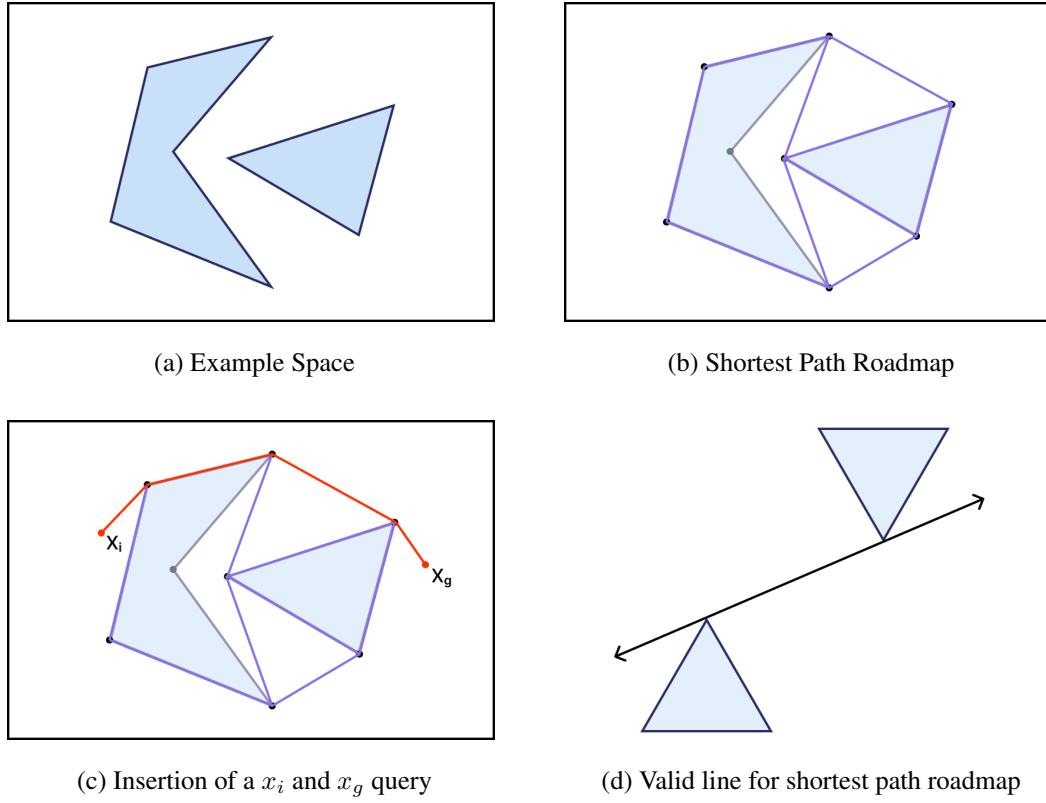


Figure 2.4: Shortest Path Roadmap Method

Figure 2.4 shows the shortest path roadmap in 2D space. A roadmap is built by identifying vertices that the line between them and the extension of that line is not in the obstacle space, [11]. For example, valid vertices and lines are shown in figure 2.4b over an example space in figure 2.4a.

Insertion of an initial state,  $x_i$ , and goal state,  $x_g$ , and its solution is shown in figure 2.4c. First, the  $x_i$  and  $x_g$  are connected to the possible vertices in the roadmap, and then the solution is found with a search method. However, optimality and efficiency depend on the selected search method.

This method can perform well in 2D space filled with low obstacles, like Vertical Cell Decomposition. However, in 3D, defining the vertices of the obstacles and calculating the decomposition is extremely difficult.

### 2.2.3 Sampling-Based Planning

Different from Combinatorial planning methods, sampling-based planning methods avoid explicit construction of the obstacle space since, in most problems, explicit construction is impractical. Sampling-based planning methods sample from the space and apply collision checks to sample points. The collision detection module does collision check operations. In sampling-based planning methods, the collision detection module is treated separately since the collision detection module is highly dependent on the geometric and kinematic models used to define the state-space of the problem, [6]. However, collision check is accepted as the most expansive calculation for every sampling-based planning method. Thus, avoiding unnecessary collision checks during the planning is favorable. Unlike other approaches, one of the main advantages of the sampling-based planning methods is being effective at high dimensional problems.

Sampling-based planning methods are resolution and probabilistic complete, which means if enough samples are taken from space, the method can find a solution if one exists and state if there is no solution to the problem. However, there is a trade-off in sampling-based planning. The high number of space samples means a slower search due to the high number of collision checks, and lower space samples mean possible missed solutions. Sampling-based planning methods can be categorized as input or sampling-based. In input-based, they can be classified as **Single-Query** and **Multi-Query**. In sampling-based, they can be classified as **Incremental Sampling** and **Batch Sampling**.

#### 2.2.3.1 Sampling Methods

Different sampling approaches can highly affect the performance of the sampling-based planning methods. In this section, the primary sampling approaches are explained.

**Random-Uniform Sampling** Random-uniform sampling of the problem space is defined as taking uniform random sample points from all over the space. Random-

uniform sampling is the most common and easiest sampling method. However, users should be careful about randomness.

**Low-Dispersion Sampling** Low-dispersion sampling aims to make the largest uncovered area as small as possible. Grid-based sampling offers the lowest dispersion.

**Low-Discrepancy Sampling** Low-discrepancy sampling aims to make every subspace have enough amount of samples.

**Boundary Sampling** Boundary sampling aims to sample from areas close to obstacles in the space. It can be accomplished by sampling from the obstacle space and then finding sampling points in the free space by going in different directions from the sample in the obstacle space.

**Bridge Test Sampling** Bridge test sampling aims to sample from areas between obstacles to represent the connectivity of the space.

**Medial-axis Sampling** Medial-axis sampling aims to sample from the medial-axis of the space. Medial-axis sampling is one of the most challenging sampling methods since constructing the medial-axis is already complex.

There are two main branches of the sampling-based planning methods in terms of processing samples.

1. **Incremental Sampling:** Samples are processed one by one. A new node is added to the search by processing a new sample node.
2. **Batch Sampling:** Samples are processed as a batch. In order to add a new point in the search, a new batch should be created and computed again. However, unlike incremental sampling, batch sampling provides parallel computing, which boosts the performance of the algorithms.

### 2.2.3.2 Single-Query Models

Single-query models do not perform pre-processing over space or samples. They are applied for a specific  $(x_i, x_g)$  pair. The general methodology can be summarized as,

- Define  $x_i$  and  $x_g$
- Initialize a graph as  $G = (V, E)$ , where  $V$  contains sample points and  $E$  contains no edge initially
- Choose a vertex  $V_{expansion}$  in  $V$  for expansion
- For a sample point  $V_{sample}$  in free space, if edge  $V_{expansion} \rightarrow V_{sample}$  is feasible, insert edge to  $E$  and  $V_{sample}$  in  $V$  if it's not contained
- Check if the solution path found or execution criteria are met, if not continue to expand

In most of the single-query methods, a tree structure is built. A solution is found when a branch of the tree reaches the goal. Trees might be single or more. Single-tree methods are called unidirectional. Methods with two trees, one initialized from the initial point and one initialized from the goal point, are called directional methods. If the method has more than two trees, it is called a multi-directional method. Methods with more than one tree can explore search space faster. However, connecting the trees is challenging. Moreover, since tree structure is suitable for complex dynamics, single-query methods can perform well even under complex dynamics. Discretization of the space as grids and checking the feasibility of the edges can solve many problems. However, this approach suffers from the resolution issues of the grids. High resolution makes the search process very slow; low resolution might miss the solution. Moreover, grids suffer from the **curse of the dimensionality** since the number of grids can be expressed as  $n^d$  where  $n$  is the number of grids in one axis, and  $d$  is the dimension of the space. There are two main approaches to overcome this property,

- Start with a resolution and increase the resolution recursively
- Sample from continuous space, just like *RRT*, *RRT\**, *FMT\**, *Informed RRT\**, *BIT\**

**Rapidly exploring random tree, RRT** Rapidly Exploring Random Trees (RRT) is a popular algorithm used in robotics and motion planning to efficiently explore and find feasible paths in high-dimensional and complex configuration spaces. The main idea behind RRT is to generate a random tree-like structure by incrementally growing a tree from an initial configuration towards the goal configuration in the configuration space, [5]. Although RRT can perform well in some applications, it cannot find the optimal solution.

1. Create a tree with a single node, which is the initial point,  $x_i$ .
2. For each iteration, sample a point from the space and find its nearest node or edge in the existing tree. Expand the tree if the path between the sample point and its nearest neighbor in the tree is feasible. If the path is not feasible, steer the sample point towards the nearest neighbor until the path is feasible.
3. Repeat step 2 until either the goal,  $x_g$ , is close enough to the tree or specific termination criteria are met.

**RRT\*** RRT\*, Rapidly exploring random tree star, aims to find optimal solutions by using the primary approach of the RRT. Optimal solutions are achieved by checking additional neighbors in the tree while inserting a new sample point instead of directly connecting the new sample point to the nearest neighbor. If any of the other neighbor nodes offers a better cost-to-arrive for a new sample point, then that tree is expanded from that node. Additional neighbors can be checked as **k-nearest** or **in a radius from the sample point**, [12].

**Fast Marching Tree, FMT\*** FMT\*, Fast Marching Tree, aims to eliminate collision checks in high dimensional and obstacle-rich environments. It uses disc-connected graphs to gradually expand outwards from  $x_i$  to the space, [13].

**Informed RRT\*** Informed RRT\* is an extension of the RRT\* algorithm that incorporates a heuristic function, represented by admissible ellipsoids, to guide the tree expansion and improve the efficiency of the planning. The key idea behind Informed

RRT\* is to use the heuristic function to bias the sampling process towards regions of the configuration space that are more likely to lead to the optimal path, [14].

**Batch Informed Trees, BIT\*** Batch Informed Trees, BIT\*, incrementally build a roadmap while maintaining an implicit random geometric graph. BIT\* uses a heuristic guidance mechanism to guide the sampling process towards the goal configuration, effectively reducing the exploration effort. BIT\* processes samples in batches, allowing re-optimization and roadmap updates, [15].

### 2.2.3.3 Multi-Query Models

Multi-Query models perform pre-processing over space or samples to create a graph for repetitive search queries. They are applied for any  $(x_i, x_g)$  pair. The general methodology can be summarized as,

- Pre-process the space information
- Build a roadmap that represents the connectivity of the space
- Define  $x_i$  and  $x_g$ , and their connections to the roadmap
- Apply a search method to find the path contained in the roadmap

In most of the multi-query methods, a roadmap is built. A solution for any  $(x_i, x_g)$  pair can be found after connecting  $(x_i, x_g)$  to the roadmap and searching for a solution. Some examples of multi-query planning methods are PRM, PRM\*, and VRM.

**Probabilistic Roadmap, PRM** Probabilistic Roadmap, PRM, creates a roadmap, which is a graph representation of the state space through random sampling. This roadmap serves as a roadmap or a connectivity graph between different states in the space. The roadmap construction is probabilistic, and the algorithm does not guarantee completeness or optimality, but it is highly efficient and widely used in practice.

1. Sample from the state space. If the sample is from obstacle space, discard the sample and take a new sample.

2. Check the neighbors of the new sample point, k-nearest, or a defined radius distance. If there is a feasible path, add an edge to the roadmap. If not, only add a sample node to the roadmap.
3. Terminate the roadmap construction process when the specified termination condition is met.
4. Find the path from  $x_i$  to  $x_g$  by performing a graph search on the constructed roadmap.

PRM constructs the roadmap in the form of a graph. The graph nodes correspond to the sampled states, and the edges represent the feasible connections between these states, [4].

**Probabilistic Roadmap Star, PRM\*** Probabilistic Roadmap Star, PRM\*, is an extension of the basic Probabilistic Roadmap, PRM, algorithm that provides probabilistic guarantees on the optimality and completeness of the generated paths. PRM\* aims to combine the efficiency of PRM in exploring high-dimensional configuration spaces with the optimality in the results. PRM\* achieves this by defining a neighborhood based on a radius function,  $r(n)$ , where  $n$  is the current number of nodes in the roadmap. The  $r(n)$  value is inversely proportional with the  $n$ , so PRM\* connects nodes with higher distances at the beginning of the constructing roadmap process and lower distances at the end, [12].

**Visibility Roadmap, VRM** Visibility Roadmap, VRM, defines the neighborhood as the visibility. VRM is based on two definitions, **Guard** and **Connector**. A sampling node becomes a guard if it sees no other guard. A sampling node becomes a connector if it can see at least two guards from different connected components in the end roadmap constructed with the guards, connectors, and their connections, [16]. VRM can build roadmaps with fewer nodes and connections since it uses visibility criteria instead of k-nearest neighbors or radius distance neighbor approach. However, since guard selection depends highly on the sampling, VRM rarely returns the optimal solutions.

#### 2.2.4 Discrete Optimization

In discrete optimization, the goal is to find the best combination of values for the decision variables from a discrete set that optimizes a given objective function. The objective function is a mathematical function that needs to be minimized or maximized, representing the goal or criteria that must be satisfied. The decision variables are the variables that we can control or manipulate to achieve the optimal solution. Discrete optimization problems are often computationally challenging, especially for large problem instances, due to the discrete nature of the decision variables and the combinatorial nature of the feasible solutions.

**Branch and Bound** Branch and Bound is an algorithmic technique used to solve optimization problems. It is designed to find the optimal solution by systematically exploring the search space while effectively pruning branches that cannot lead to better solutions than the currently known best solution, [17]. The algorithm initially finds a solution in a search space by reaching a leaf node and calculates the cost of the solution. After that, the algorithm continues to explore the search space. However, if the current branch in the search space loses the possibility to give a better solution than the currently best solution, then that branch is pruned. The initial solution can be combined with the heuristic methods to find a better solution to prune more branches.

**Constrained Programming** Constrained programming aims to find the best solution to an optimization problem subject to constraints. In constrained programming, the goal is to maximize or minimize an objective function while satisfying a set of constraints that limit the possible values of the decision variables. Constrained programming limits search space by using constraints, [18]. It is very effective at finding if a feasible solution exists for a problem. Moreover, constrained programming is also very effective at optimizing small search spaces.

**Local Search** Local search is an optimization technique used to find better solutions to an optimization problem by iteratively exploring the neighborhood of a current solution. Instead of considering the entire search space, local search focuses on a

particular region around the current solution and attempts to move to a better solution within that local region. Local search is handy for solving large-scale and discrete optimization problems where the entire search space is too vast to explore exhaustively. Local search selects the best neighboring solution based on an objective function and can find good solutions. However, it can get stuck at the local optimum and might not be able to find the global minimum.

**Linear Programming** Linear Programming is based on representing the state space, constraints, and objective function as linear equations. The feasible region can be found at the intersection of the constraint inequalities, and the optimal solution can be found at one of the corners of the feasible region, [19].

**Genetic Algorithms and Meta-heuristics** Genetic Algorithms and Meta-heuristics are high-level general problem-solving techniques. They can handle complex optimization problems. They are flexible and adaptable methods designed to be efficient regardless of the problem. Many of them are inspired by biological processes and natural phenomena. They might not find the global optimal solutions. Moreover, their performance highly depends on the algorithm parameters and problem formulation, [20].

**Large Neighborhood Search, LNS** Large Neighborhood Search, LNS, is known for its effectiveness in finding high-quality solutions for large and complex combinatorial optimization problems. By exploring diverse solutions in a large neighborhood, LNS can efficiently navigate the search space and avoid getting trapped in local optima. The main process of the problem can be expressed as, [21],

1. Find a feasible solution, might use constrained programming
2. Select a subset of decision variables and perturb them
3. Perform a local search to find the best solution in the selected subset
4. Select if acceptance criteria are satisfied based on conditions and probability

5. Repeat steps 2-4 until the termination criteria are met

One of the main advantages of the LNS is that it can be applied for any period. Solution quality will improve for higher LNS run time. However, it can still offer feasible and reasonable solutions for short run-time.



## CHAPTER 3

### METHODOLOGY

In this chapter, the methodology of this thesis study is explained. Leading planning and discrete optimization methods in the literature are shared in chapter 2. Discrete optimization algorithms in the literature can be used to find the optimal solution in a dynamic optimization problem since discrete optimization problems can find the optimal solution in a set of possible solutions. However, we need a new approach to planning algorithms to find a set of possible solutions mainly because planning algorithms in the literature, regardless of being multi-query or single-query, are only interested in finding a single path from  $x_i$  to  $x_g$ . In DOPs, alternative paths from  $x_i$  to  $x_g$  should be known since the active state of the world might lead us to use an alternative path. Thus, we need a new roadmap or tree approach that provides,

- Alternative paths from  $x_i$  to  $x_g$
- Optimal, not redundant, solutions of alternative paths
- Not requiring a new tree or roadmap for constant  $x_i$  and variable  $x_g$

#### 3.1 Building Paths

Assume we built a roadmap in a  $C$  as shown in figure 3.1. Neighborhood is defined as visibility, and an edge is added if any two nodes have a feasible path between them. Let the initial state,  $x_i$ , be **Node 0**. Assume that only one goal state,  $x_g$ , is defined at **Node 6**.

At first, try to create paths with classical BFS, where visited nodes are marked and not revisited. Results of the classical BFS approach over figure 3.1 are shown in figure

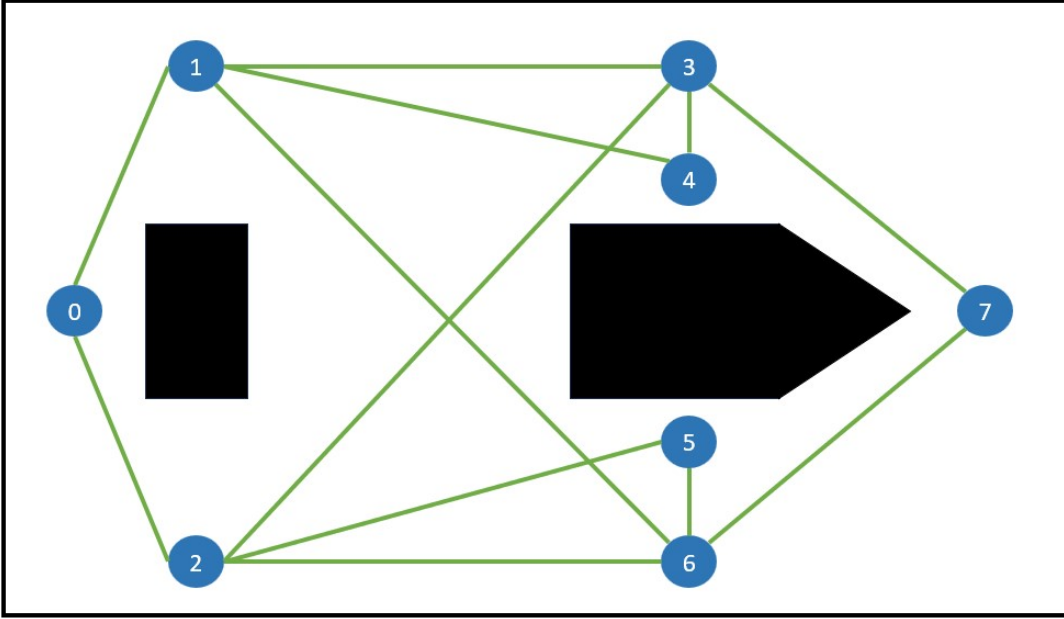


Figure 3.1: An example roadmap built-in  $C$

3.2. This result is not satisfying for DOPs since the search could not find alternative paths branching from **Node 2**. Marking nodes as visited and removing them from the search space results in losing information about alternative paths. Thus, let us try BFS again without marking the nodes as visited so the algorithm can revisit each node even if it has already been visited. The search results are shown in figure 3.3. The search approach found the alternative paths. However, the search also found redundant paths, shown as red paths in figure 3.3. We designate these paths as redundant due to the presence of at least one node within each path that can be omitted while maintaining the feasibility of the path. In both figure 3.3b and figure 3.3c, a complete search could not be shown due to the high amount of paths. The main problem is not marking nodes as visited and revisiting them. Even though the search can find alternative paths in the space, it also finds many redundant paths, making the method impractical.

In order to find alternative paths in the space and eliminate the redundant paths, we need a new approach. First, let us make a definition that helps us to find alternative paths.

**Lemma 3.1.1.** *A path is **sub-optimal** if any of the subsets, including its first and last node, of the path is not feasible.*

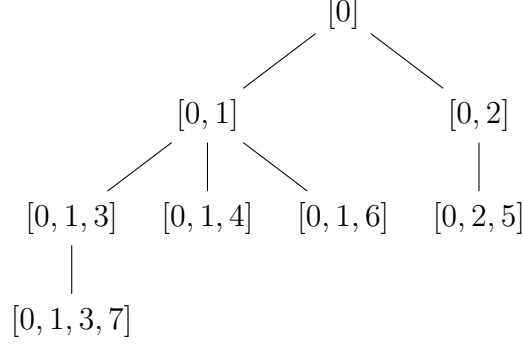


Figure 3.2: BFS Search over Figure 3.1

**Corollary 3.1.1.1.** *A path with two or fewer nodes is **sub-optimal** if the path is feasible. A path with more than two nodes is **sub-optimal** if any of the subsets, with two or more nodes while including the path's first and last node excluding itself, of the path is not feasible.*

Corollary 3.1.1.1 implies that if there is a feasible path between **Node A** and **Node B**, a path such as {**Node A**, **Node C**, **Node B**} is not **sup-optimal** since **Node C** is redundant. Creating only the **sub-optimal** paths means eliminating the redundant ones.

**Corollary 3.1.1.2.** *A path is **sub-optimal** if its parent is **sub-optimal** and the last node of the path only can be seen by **penultimate** node of the path.*

*Proof.* Assume a parent path,  $P_{parent} = \{N_0, N_1\}$  with two nodes that are feasible and sub-optimal from the definition at lemma 3.1.1.1. Assume a node  $N_2$  can be seen from  $N_1$  but not from  $N_0$ . Child path,  $P_{child} = \{N_0, N_1, N_2\}$  is feasible since only consecutive nodes can see each other. Moreover, the child path's only valid subset for checking sub-optimality is  $P_{subset} = \{N_0, N_2\}$ , and it is not feasible since  $N_2$  cannot be seen by  $N_0$ . Thus,  $P_{child}$  is sub-optimal. Moreover, adding another node,  $N_3$ , can only be seen from  $N_2$ , creates a sub-optimal child since any of the subsets of the  $P_{child}$  disrupt the sequential visibility of consecutive nodes.  $\square$

Figure 3.4 shows that BFS with only **sub-optimal** paths. The sub-optimality approach helps search to find all alternative paths while preventing the search from finding

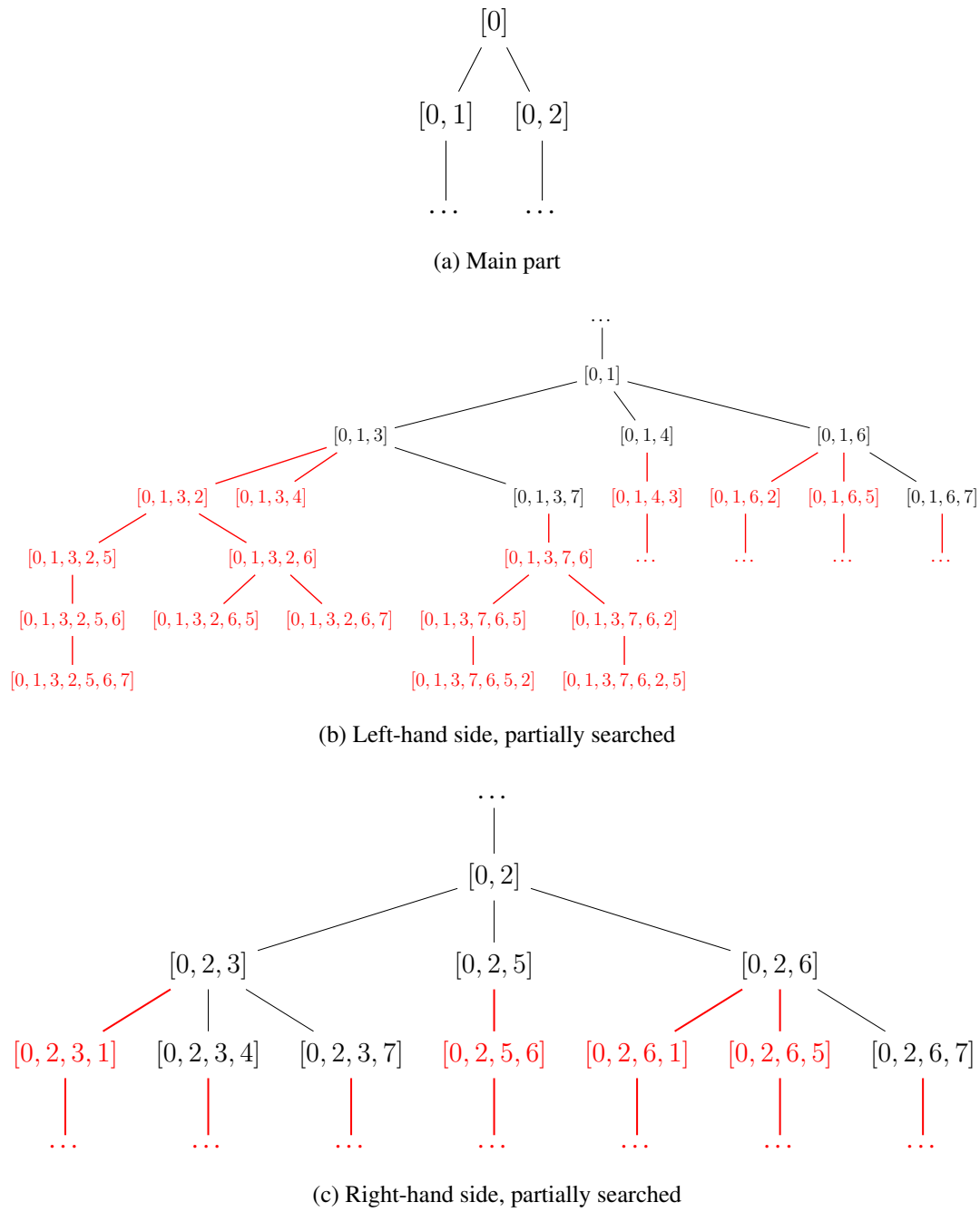


Figure 3.3: BFS Search without marking nodes as visited over figure 3.1, redundant paths colored with red

redundant paths. Most of the search space is pruned in figure 3.4 compared to figure 3.3. Pruning the search space is crucial at the densely sampled search spaces since the sample number of states can be enormous, and this leads to even more paths.

### 3.2 POPS: Possible Optimal Paths Search

In this section, we will define our new search algorithm **Possible Optimal Paths Search, POPS**, to find possible optimal paths under dynamic objective functions. Dynamic objective functions mean that the weight of the edges or nodes can be changed dynamically over a graph. **POPS** is a search method to find any path from a single source that can be optimal for a given set of weights to edges or nodes. **POPS** has only one requirement, which is that the graph should hold the triangle inequality, which means that for any nodes,  $Node_1$ ,  $Node_2$ , and  $Node_3$ , in a graph,

$$d(Node_1, Node_3) \leq d(Node_1, Node_2) + d(Node_2, Node_3) \quad (3.1)$$

Fortunately, this requirement can be avoided by defining a new node,  $Node_4$ , between  $Node_1$  and  $Node_3$  to eliminate the direct connection of the  $Node_1$  and  $Node_3$ .

**POPS** ignores the current weights of the graph and focuses on finding all alternative paths that can be optimal for at least one set of weights of the edges or nodes by exploring the graph from a source node.

The **POPS** 's fundamentals are explained in section 3.1. **POPS**, basically finds **sub-optimal paths** in a graph. Sub-optimality is explained at lemma 3.1.1 and corollary 3.1.1.1. The algorithm is shared in figure 3.5. When the **POPS** got the sub-optimal paths for an initial source node, every found sub-optimal path is saved in the reached nodes. Then, the optimal path to the goal node can be found by applying the current weights of the nodes or edges to the saved paths in the goal node. Note that the result of the **POPS**, for the figure 3.6, is the same as the tree shared in figure 3.4.

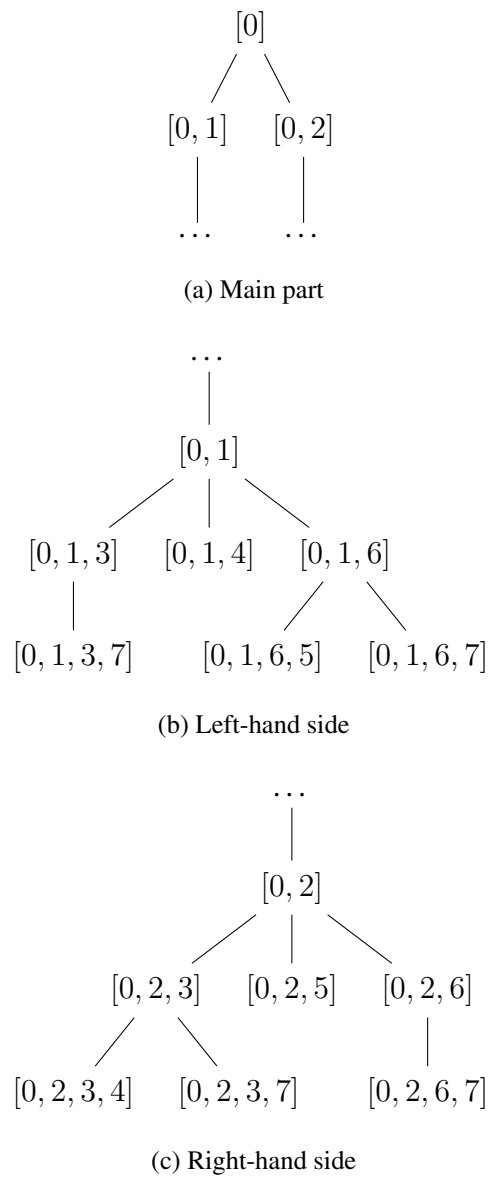


Figure 3.4: BFS Search with only **sub-optimal** paths over figure 3.1

**Require:** Graph  $G(V, E)$  where  $V$  is the set of nodes,  $E$  is the set of edges

**Ensure:**  $d(N_1, N_3) \leq d(N_1, N_2) + d(N_2, N_3)$  for  $N_1, N_2, N_3 \in V$

```
1: Select a source node,  $N_i$ 
2: Define root path,  $P_{root} = \{N_i\}$ 
3: Initialize open path list,  $OPL = [P_{root}]$ 
4: while  $OPL \neq []$  do
5:    $P_{current} = OPL[0]$ 
6:   for  $node$  in  $V$  do
7:      $P_{candidate} = P_{current} + \{node\}$ 
8:     if  $P_{candidate}$  is feasible then
9:       if  $P_{candidate}$  is sub-optimal then
10:         $node.paths \leftarrow P_{candidate}$ 
11:         $OPL \leftarrow P_{candidate}$ 
12:       end if
13:     end if
14:   end for
15: end while
```

Figure 3.5: **POPS** algorithm

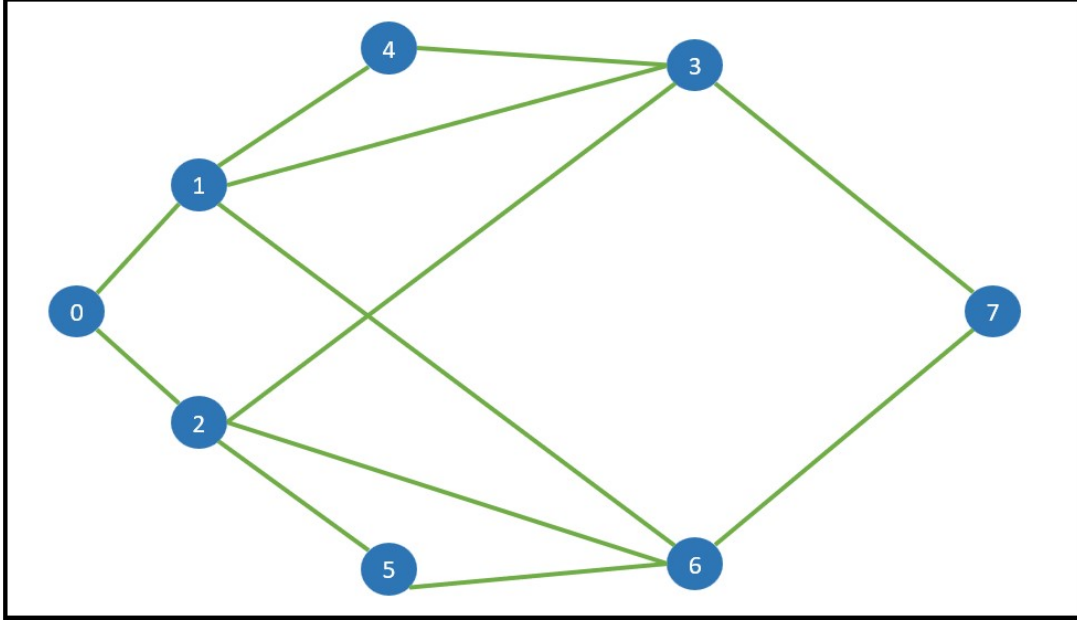


Figure 3.6: An example graph to show **POPS**

### 3.3 Building Roadmap

Figure 3.4 shows that the sub-optimality approach helps to prune the vast majority of the search space compared to figure 3.3. Pruning the search space without losing the alternative path information is important for the many other dynamic optimization problems. However, let us check the leaf paths, leaf nodes in the figure 3.4 search tree. In planning problem definition, there is an initial state,  $x_i$ , goal state,  $x_g$ , pair, and solution is a set of states that lead  $x_i$  to  $x_g$ . In chapter 2, we also observed that multi-query methods build the roadmap without caring about the  $x_i$  and  $x_g$  information. In contrast, single-query methods build the tree structure always with the  $x_i$  and sometimes with the  $x_g$  information. The goal state,  $x_g$ , information might change during the solution phase of the dynamic optimization problem, or a new goal state might be defined. Moreover, changing the objective function or decision variables of the discrete optimization problems might result in a different optimal path from  $x_i$  to  $x_g$ . In order to handle these situations, we aim to identify the critical sub-regions within the space, from which the optimal solution could potentially include states. In the case study problem, these sub-parts will be the positions to place drones. Leaf paths, leaf nodes at the figure 3.4 search-tree, do not add information about critical

space parts since they do not produce sub-optimal paths. For example, assume a new  $x_g$  is defined at **Node 7**, and we want to find proper paths to reach the new  $x_g$ . Path  $\{0, 1, 3\}$  could be used to reach the goal state by reaching **Node 1** and then **Node 3**, path  $\{0, 1, 6\}$  could be used to reach the goal state by reaching **Node 1** and then **Node 6**, path  $\{0, 2, 3\}$  could be used to reach the goal state by reaching **Node 2** and **Node 3** or path  $\{0, 2, 6\}$  could be used to reach the goal state by reaching **Node 2** and **Node 6**, since last node of those paths can see the **Node 7**. However, any of the leaf paths ending with **Node 7** are not proper since there is not a goal state, defined at any node of the figure 3.1, that agent must reach the **Node 7** to reach the goal state,

**Lemma 3.3.1.** *A path is a **proper path** if it can produce at least one **sub-optimal** child path.*

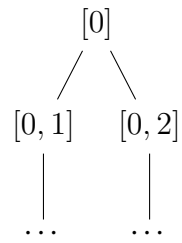
**Lemma 3.3.2.** *A path is a **complete path** if it is a **proper path** and any of its child paths are not a **proper path**.*

**Theorem 3.3.3.** *If a roadmap is built with a set of all nodes contained at the complete paths, that roadmap can lead to any goal state,  $x_g$ , in the continuous  $C_{free}$ , from  $x_i$ .*

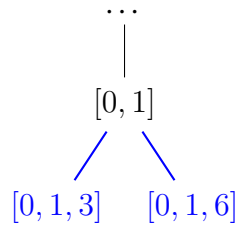
*Proof.* Assume for the completed search of a continuous  $C_{free}$  there are  $n$  **complete** paths found at a densely sampled, all the connectivity of the  $C_{free}$  is represented by the sampled nodes,  $C_{free}$  and a roadmap,  $R$ , is built with the set of all nodes contained in the **complete paths**. Assume a node,  $N_{goal}$ , is not reachable from the roadmap.  $C_{free}$  is continuous. Thus, at least one sample node,  $N_{middle_0}$ , can see  $N_{goal}$ . If  $N_{middle_0}$  can be seen from any of the paths,  $P_{any}$ ,  $P_{any} + \{N_{middle_0}\} + \{N_{goal}\}$  is a **sub-optimal** path since last node of  $P_{any}$  can not see  $N_{goal}$ . Thus,  $P_{any} + \{N_{middle_0}\}$  is a **proper path** since it can produce a **sub-optimal** child as  $P_{any} + \{N_{middle_0}\} + \{N_{goal}\}$ .  $P_{any}$  has a **proper** child, so it cannot be a **complete** path. Thus, there is not a node,  $N_{goal}$ , such that it cannot be seen by any of the **complete** paths. A roadmap is built with all nodes contained at **complete** paths. Thus, there is not a node,  $N_{goal}$ , such that the roadmap cannot see it.

□

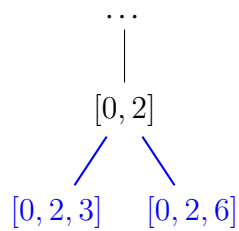
The figure 3.7 BFS with only **proper** paths are shown. Compared to figure 3.4, the number of paths is decreased, which means search space is pruned even more.



(a) Main part



(b) Left-hand side



(c) Right-hand side

Figure 3.7: BFS Search with only **proper** paths over figure 3.1, complete paths colored with blue

Figure 3.8 shows the roadmap built with complete paths. Our approach eliminated 3 points from the original roadmap, 3.1, since they do not provide any information about crucial spaces to reach all of the  $C_{free}$  from the initial source  $Node_0$ .

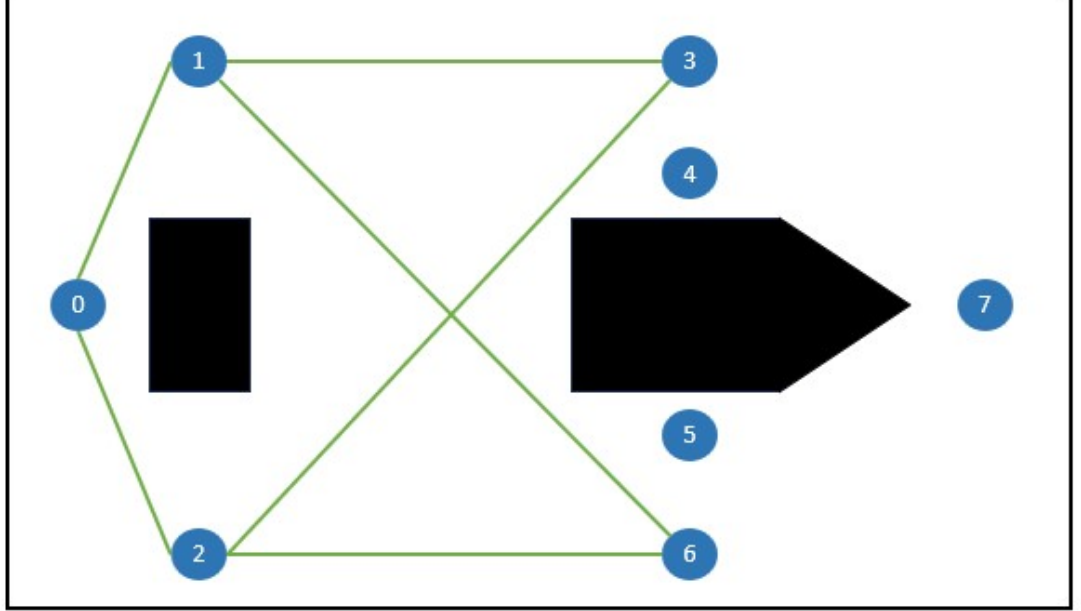


Figure 3.8: The roadmap built with complete paths in  $C$

### 3.4 Algorithm Design

The pseudo-code for finding complete paths is presented in the figure 3.9. Between line 6 to line 14 algorithm finds sub-optimal paths of the currently inspected path. Between line 15 to line 24 paths, the algorithm checks whether found sub-optimal paths are also proper paths; if they are, the algorithm creates these paths. At line 9 and line 18, the algorithm applies **collision check**, the most expansive operation. However, in the line 7 and line 16 algorithm iterate over all  $N \in V$ , and it applies **collision check**. This iteration is inefficient since, while building the paths algorithm can save the information of seen nodes by the path and check for only not seen nodes in  $V$ . In order to achieve that, we will define an adjacency vector for the path, and the algorithm will only check if a node in  $V$  is stated as not seen in the adjacency vector. We will call that adjacency vector as **seen nodes vector** for ease of referencing. Moreover, by definition of paths, they only expand through their last node's visibility

**Require:** Densely sampled continuous  $C_{free}$  where  $V$  contains all samples

**Ensure:**  $\forall N \in C_{free}$  for  $N \in V$

```

1: Pick a random sample point,  $N_i$ 
2: Define root path,  $P_{root} = \{N_i\}$ 
3: Initialize open path list,  $OPL = [P_{root}]$ 
4: while  $OPL \neq []$  do
5:    $P_{current} = OPL[0]$ 
6:   Initialize candidate path list,  $CPL = []$ 
7:   for  $node$  in  $V$  do
8:      $P_{candidate} = P_{current} + \{node\}$ 
9:     if  $P_{candidate}$  is feasible then
10:      if  $P_{candidate}$  is sub-optimal then
11:         $CPL \leftarrow P_{candidate}$ 
12:      end if
13:    end if
14:  end for
15:  for  $P_{candidate}$  in  $CPL$  do
16:    for  $node$  in  $V$  do
17:       $P_{next} = P_{candidate} + \{node\}$ 
18:      if  $P_{next}$  is feasible then
19:        if  $P_{next}$  is sub-optimal then
20:           $OPL \leftarrow P_{candidate}$ 
21:        end if
22:      end if
23:    end for
24:  end for
25: end while

```

Figure 3.9: Algorithm 1, Find Complete Paths

set. This information is captured at 19, and we can use this information to improve the algorithm to reduce the number of collision checks.

**Lemma 3.4.1.** *Every path has a **seen nodes vector**, a  $m$  dimensional bit array where  $m$  is the number of the sample points in  $C_{free}$ . 1 represents a node seen by a node in the path, and 0 means a node cannot be seen by any node in the path.*

The revised algorithm with lemma 3.4.1 is shown in figure 3.10 pseudo-code. Between line 2 and line 8, the algorithm finds possible candidates for the root path and then starts to expand paths. At line 17, **seen nodes vector** is used to eliminate unnecessary collision checks. Moreover, at line 21, the path's last node's visibility set is saved into the path since that information is helpful for the next level expansion of the path.

### 3.5 Improving Algorithm's Performance

In section 3.4, the basic design of the algorithm is shared. However, real-life implementation of the basic design of the algorithm faced some problems and showed poor performance. This section explains applied methods to solve those problems and improve the overall performance.

The algorithm shared at figure 3.10 suffered from,

1. Excessive number of complete paths
2. Excessive number of roadmap nodes due to the excessive number of complete paths
3. Impractical or inefficient run-time at obstacle-rich environments

The first and second items can be solved by decreasing the number of complete paths. In order to understand the problem clearly, we can inspect figure 3.12. In figure 3.12, the green node is the initial node that produces the root path, and two purple nodes are shown. Both purple nodes can see many nodes that cannot be seen from the root

**Require:** Densely sampled continuous  $C_{free}$  where  $V$  contains all samples

**Ensure:**  $\forall N \in C_{free}$  for  $N \in V$

```

1: Pick a random sample point,  $N_i$ 
2: Define root path,  $P_{root} = \{N_i\}$ 
3: for  $node$  in  $V$  do
4:    $P_{candidate} = P_{root} + \{node\}$ 
5:   if  $P_{candidate}$  is feasible then
6:      $P_{root.candidates} \leftarrow P_{candidate}$ 
7:   end if
8: end for
9: Initialize open path list,  $OPL = [P_{root}]$ 
10: while  $OPL \neq []$  do
11:    $P_{current} = OPL[0]$ 
12:   for  $candidate$  in  $P_{current.candidates}$  do
13:      $P_{current.seen\_nodes\_vector}[candidate] = 1$ 
14:   end for
15:   for  $P_{candidate}$  in  $P_{current.candidates}$  do
16:     for  $node$  in  $V$  do
17:       if  $P_{current.seen\_nodes\_vector}[node] == 0$  then
18:          $P_{next} = P_{candidate} + \{node\}$ 
19:         if  $P_{next}$  is feasible then
20:           if  $P_{next}$  is sub-optimal then
21:              $P_{candidate.candidates} \leftarrow P_{next}$ 
22:              $OPL \leftarrow P_{candidate}$ 
23:           end if
24:         end if
25:       end if
26:     end for
27:   end for
28: end while

```

Figure 3.10: Algorithm 2, Find Complete Paths with **seen nodes vectors**

path. Thus, they are eligible as candidates. However, they are incredibly close and produce almost similar paths.

Moreover, 3 line segments, blue, red, and black, are shown in the figure 3.12. Assume we picked the closest node from the continuous part of any line. This node can see many nodes that the green node cannot see. Thus, it can produce sub-optimal paths and be a candidate for root path. However, many sample points are further from the green nodes in those lines. Due to obstacles and a high amount of samples in  $C_{free}$ , even slight distances between nodes result in new nodes that can be seen. Thus, new paths are created, and the number of paths is increasing significantly.

Assume we placed a new node, extremely close to **node 1**, to the roadmap shown in figure 3.8. The new roadmap is shown in figure 3.13. The algorithm designed in section 3.4 will produce new connections shown as orange in figure 3.13. The complete paths are formed at 3.13, shown in figure 3.14. **Node 8**, created almost same paths with **Node 1**. This phenomenon results in an excessive amount of paths in a densely sampled  $C_{free}$  and impractical algorithm performance. This phenomenon can be solved by defining a new vector at path objects. Initially, this vector is a copy of the path's **seen nodes vector**. Unlike **seen nodes vector**, this vector will be manipulated by candidates during the search. We will name that vector as **candidates common vector**.

**Lemma 3.5.1.** *Every path has a **candidates common vector**, a  $m$  dimensional bit array where  $m$  is the number of the sample points in  $C_{free}$ . 1 represents a node that can be seen by any node or candidate of the path, and 0 represents a node that cannot be seen by any node or candidate of the path. Initially, this vector is a copy of **seen nodes vector**. However, unlike **seen nodes vector**, **candidates common vector** can be accessed by any candidate of the path. During the search phase, a candidate only checks 0 nodes and marks that index with 1 if it can see that node.*

We are going to use **candidate nodes vector** stated at lemma 3.5.1 to eliminate remarkably similar paths shown at 3.13. To do this, we are going to change the **seen nodes vector** with **candidate nodes vector** at line 17. Thanks to this change, every candidate can only create a **sub-optimal** path if it can see a node that former candidates do not see. The paths created from **Node 8** at figure 3.14 are not going to

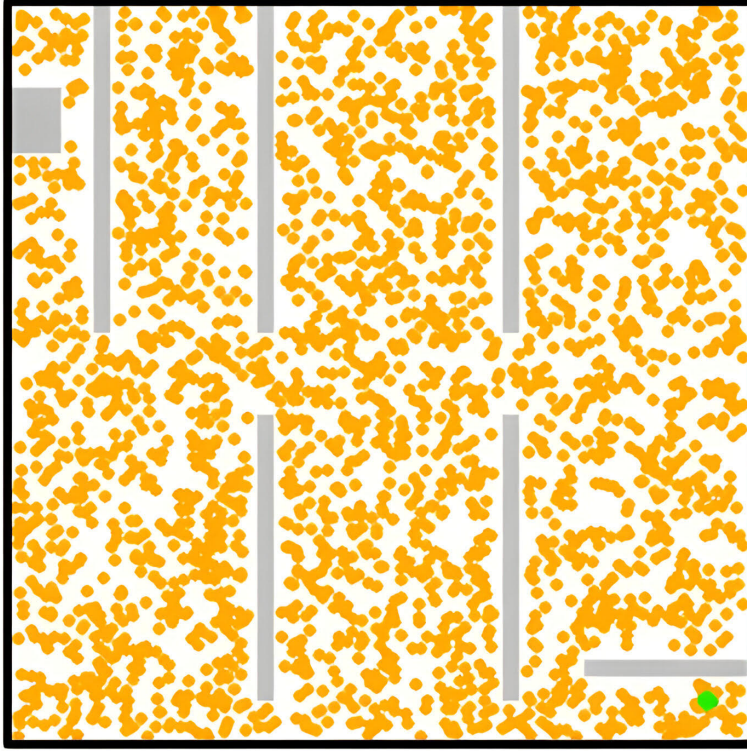


Figure 3.11: Example space densely sampled with 3000 nodes

be created now, since **Node 1** is going to mark **Node 3** and **Node 6** at **candidates common vector**. However, currently, **Node 2** also is not going to check the **Node 3** since it is marked by **Node 1**. We will backtrack previously selected candidates' seen nodes to prevent this information loss.

Another scenario increases the complete paths shared in figure 3.15. In this scenario, the visibility between **Node 1** and **Node 6** is prevented by the obstacle, unlike figure 3.13. Figure 3.16 shows the resulting complete paths. If the algorithm checked the **Node 8** before **Node 1**, it would not create any paths from **Node 1** since **Node 1** can not see any node not seen by **Node 8**. However, due to random sampling, which order algorithm will iterate over sampling nodes is unknown. This situation can happen multiple times in a densely sampled environment. In order to prevent this situation, we will use the backtracking mechanism, iteration of a candidate over previous candidates' seen nodes, to our advantage.

**Lemma 3.5.2.** *Suppose the last created candidate path can see every node, marked as 1 to the **candidates common vector** by a previous candidate path. In that case, the*

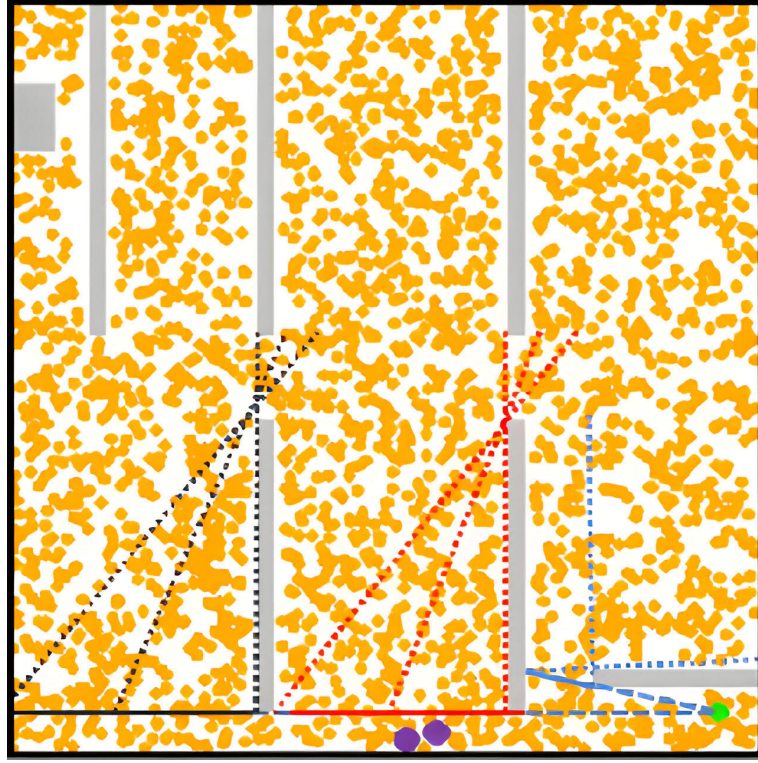


Figure 3.12: Illustration of excessive number of paths in 3.11

*last created candidate path dominates that previous candidate path. The last node of the dominated previous path is named as **dominated node**, stored at the last node of the last created candidate path, named as **dominating node**. Note that the dominated node and dominating node must see each other.*

Improved roadmap by using lemma 3.5.2, shown in figure 3.17. The resulting complete paths using lemma 3.5.2 are shown in figure 3.18. By definition, **dominated nodes** are alternative nodes to specified path nodes, which might improve the objective function. Their information is essential since they offer possible better solutions. However, they do not have to produce paths since at least one node, **dominating node**, contains their path. The modified algorithm with lemma 3.5.1 and lemma 3.5.2 is shown in figure 3.19.

Lemma 3.5.1 and 3.5.2 improve the algorithm's performance and make the algorithm more compact and efficient. However, the algorithm still shows poor, impractical, in the worst case, performance in obstacle-rich environments. First, let us understand the problem more clearly by inspecting figure 3.20. Figure 3.20 shows a densely

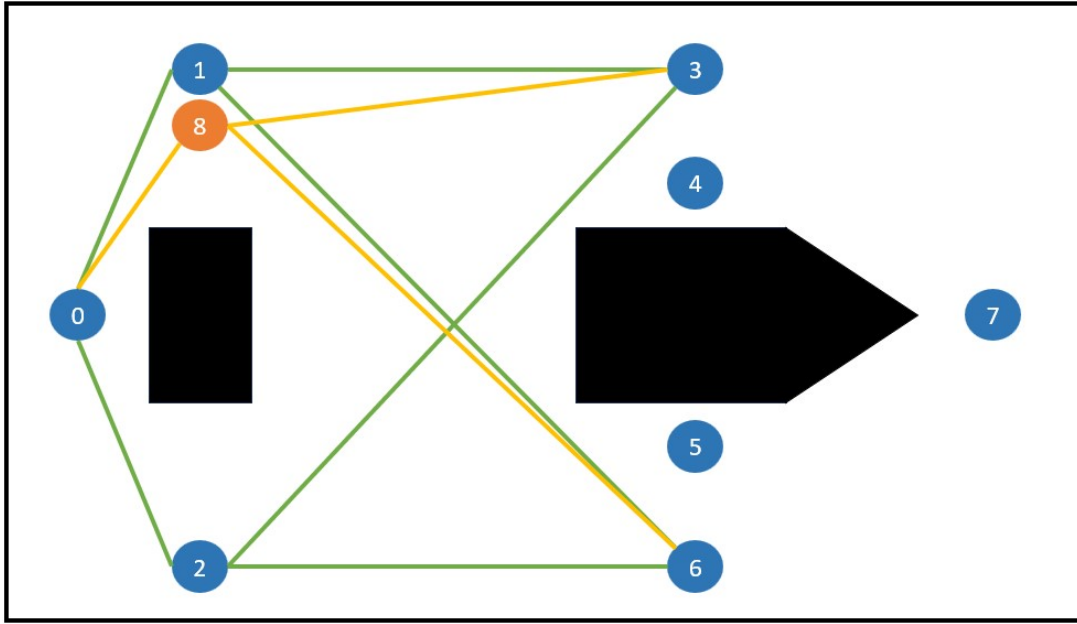


Figure 3.13: Illustration of extremely close sample nodes in  $C_{free}$

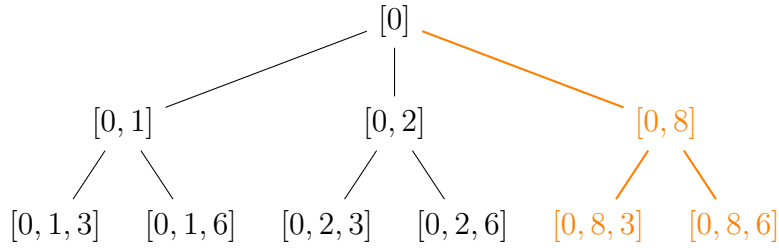


Figure 3.14: BFS Search with only **proper** paths of figure 3.13, new paths over 3.7 shown with orange

sampld obstacle-rich environment. In figure 3.21, the main paths, going to be created by the algorithm, are shared in figure 3.19. Assume half of the samples is sampled from the left side, and the other half is sampled from the right the black obstacle at 3.20. The black obstacle almost halves the space into two sub-parts, and paths will be expanded into one of those parts. In 3.21, paths are colored as blue and green. Those colors represent the specific sub-parts of the space that paths are expanding. The algorithm proposed in figure 3.19 checks every node not seen by the former nodes in the path or other candidates while expanding a path. This behavior leads to every green or blue path, shown at 3.21, checking every node at the other side of the black obstacle, even though paths can never see those nodes. This checking is the

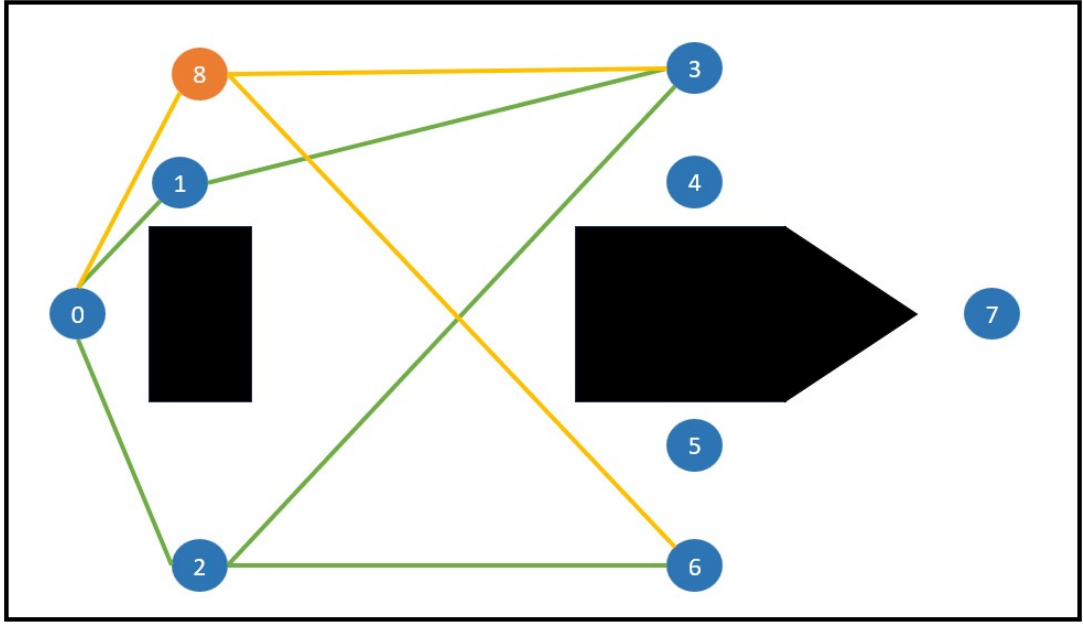


Figure 3.15: Alternative scenario for of extremely close sample nodes in  $C_{free}$

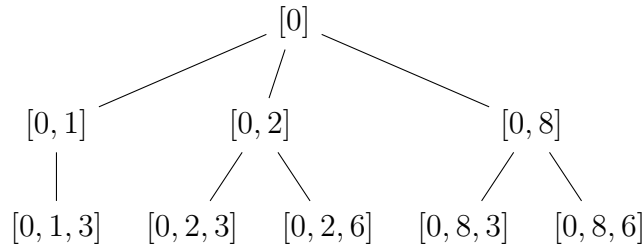


Figure 3.16: BFS Search with only **proper** paths of figure 3.15

most expansive operation, collision check, in a planning problem. Moreover,  $C_{space}$ , shown in figure 3.21, is obstacle-rich at both sides of the black obstacle. Obstacle-rich space creates many alternative paths, and each alternative path calls an excessive number of collision check operations. Thus, the algorithm shows poor performance in obstacle-rich environments.

**Lemma 3.5.3.** *If a path creates a new candidate path with a node never seen before by any path, this means the path explores the configuration space. This kind of path expansion is called **exploration**.*

**Lemma 3.5.4.** *If a path creates a new candidate path with a node, seen before by any path, this means the path connects the sub-parts of the space. This kind of path expansion is called **connectivity**.*

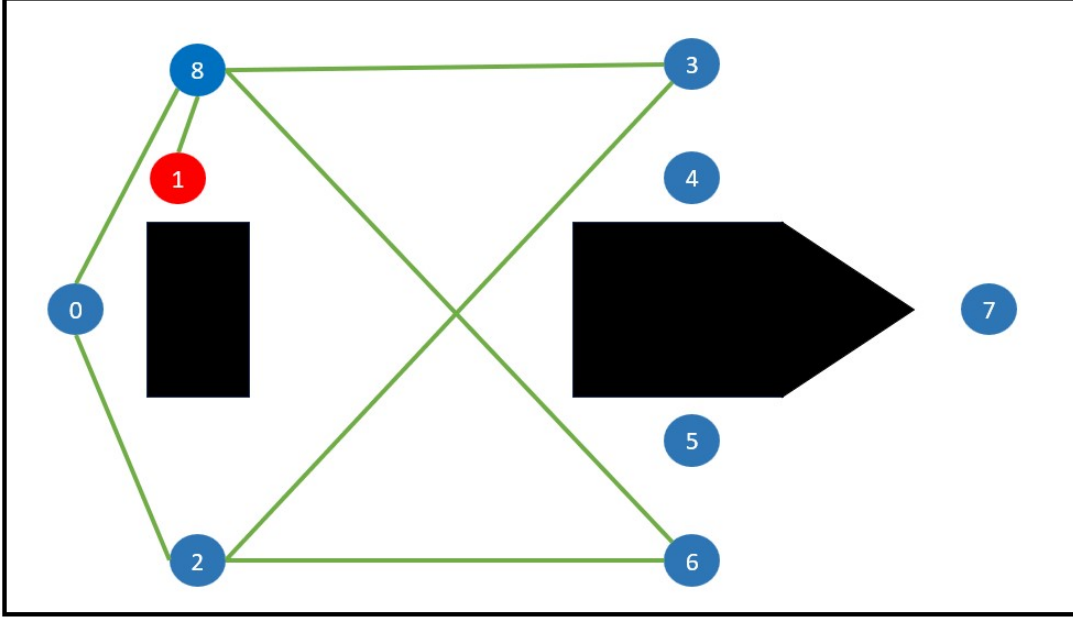


Figure 3.17: Alternative scenario for of extremely close sample nodes in  $C_{free}$ , with **dominated nodes** shown as red

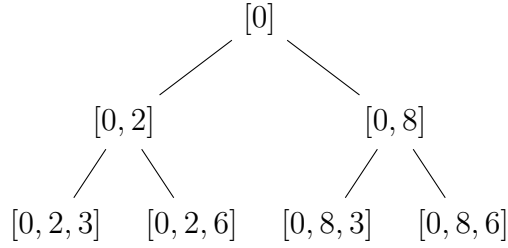


Figure 3.18: BFS Search with only **proper** paths of figure 3.17

Concepts of **exploration** and **connectivity** are defined at lemma 3.5.3 and lemma 3.5.4. Every created path performs one of these concepts. Exploration of the space is completed when all the sampled nodes in the configuration space are seen in at least one of the paths. The algorithm proposed at 3.19 completes the exploration of the space very efficiently since the algorithm constantly expands all of their paths through unexplored areas of the configuration space if possible. However, connectivity continues until every leaf path becomes a complete path.

**Lemma 3.5.5.** *Exploration of the configuration space is complete when all the sample nodes are seen by at least one path. Connectivity of the configuration space is complete when the paths become complete paths.*

**Require:** Densely sampled continuous  $C_{free}$  where  $V$  contains all samples

**Ensure:**  $\forall N \in C_{free}$  for  $N \in V$

```

1: Pick a random sample point,  $N_i$ 
2: Define root path,  $P_{root} = \{N_i\}$ 
3: for  $node$  in  $V$  do
4:    $P_{candidate} = P_{root} + \{node\}$ 
5:   if  $P_{candidate}$  is feasible then
6:      $P_{root}.candidates \leftarrow P_{candidate}$ 
7:   end if
8: end for
9: Initialize open path list,  $OPL = [P_{root}]$ 
10: while  $OPL \neq []$  do
11:    $P_{current} = OPL[0]$ 
12:   for  $candidate$  in  $P_{current}.candidates$  do
13:      $P_{current}.seen\_nodes\_vector[candidate] = 1$ 
14:   end for
15:   Initialize proper candidate list,  $PCL = []$ 
16:   for  $P_{candidate}$  in  $P_{current}.candidates$  do
17:      $add\_flag == 0$ 
18:     for  $node$  in  $V$  do
19:       if  $P_{current}.candidate\_common\_vector[node] == 0$  then
20:          $P_{next} = P_{candidate} + \{node\}$ 
21:         if  $P_{next}$  is feasible then
22:            $P_{current}.candidate\_common\_vector[node] == 1$ 
23:            $P_{candidate}.candidates \leftarrow P_{next}$ 
24:            $PCL \leftarrow P_{candidate}$ 
25:            $add\_flag == 1$ 
26:         end if
27:       end if
28:     end for

```

(a) Lines 1-28

Figure 3.19: Find Complete Paths with **candidates common vectors** and **dominated/dominating nodes**

```

29:      if  $add\_flag == 1$  then
30:          for  $former\_candidate$  in  $PCL$  do
31:               $dominated = 1$ 
32:              for  $check\_path$  in  $former\_candidate.candidates$  do
33:                   $P_{check} = P_{candidate} + \{check\_path.last\_node\}$ 
34:                  if  $P_{check}$  is feasible then
35:                       $P_{candidate}.candidates \leftarrow P_{check}$ 
36:                  else
37:                       $dominated = 0E$ 
38:                  end if
39:              end for
40:          end for
41:      end if
42:  end for
43:   $OPL \leftarrow PCL$ 
44: end while

```

(b) Lines 29-44

Figure 3.19: Find Complete Paths with **candidates common vectors** and **dominated/dominating nodes** (cont.)

Impractical or poor algorithm performance is due to the connectivity behavior of the paths at obstacle-rich configuration spaces; since the exploration of the configuration space is complete at the beginning of the search, 3rd level expansion of the search at figure 3.20. However, the knowledge that the exploration of the configuration space is complete can be used to restrict the remaining search for connectivity.

**Theorem 3.5.6.** *After the exploration of the configuration space is completed, a not-completed path might create a child path with a node not included in the roadmap at most once. After that, any child of the path can be expanded with nodes already included in the roadmap if it is not complete.*

*Proof.* Assume  $C_{free}$  is sampled densely with  $k$  nodes,

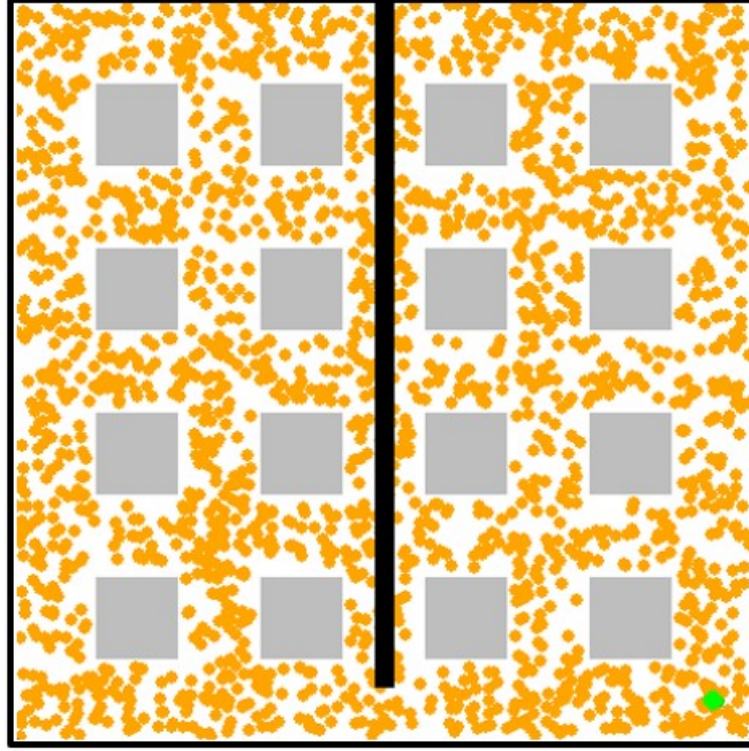


Figure 3.20: A densely sampled, obstacle-rich  $C_{space}$

- For any initial state,  $x_i$ , start root path as  $P_0 = \{x_i\}$ .
- Calculate nodes that can be reached from  $P_0$ 's last node,  $x_i$ . This set of nodes is called the last node's visibility set,  $LNV_0$ . Path's seen nodes vector,  $PSN_0$ , can be calculated as,

$$PSN_{current} = PSN_{parent} + LNV_{current} \quad (3.2)$$

For the root node,  $PSN = LNV$ .

- If any node,  $x_{any} \in LNV$ , can create a sub-optimal path as  $P_{child} = P_{parent} + \{x_{any}\}$ , where  $PSN_{parent} \subsetneq PSN_{child}$ ,  $P_{child}$  is a candidate child.
- A candidate child,  $P_{candidate}$ , of  $P_{parent}$  is selected as a child, if there is not a candidate child,  $P_{other}$ , in the candidate children of the  $P_{parent}$ , excluding  $P_{candidate}$ , has a  $PSN_{other}$  that includes  $PSN_{candidate}$ . It can be stated as,

$$PSN_{child} \not\subset PSN_{other} \quad (3.3)$$

for  $\forall P_{other} \in P_{parent}.candidates - \{P_{child}\}$

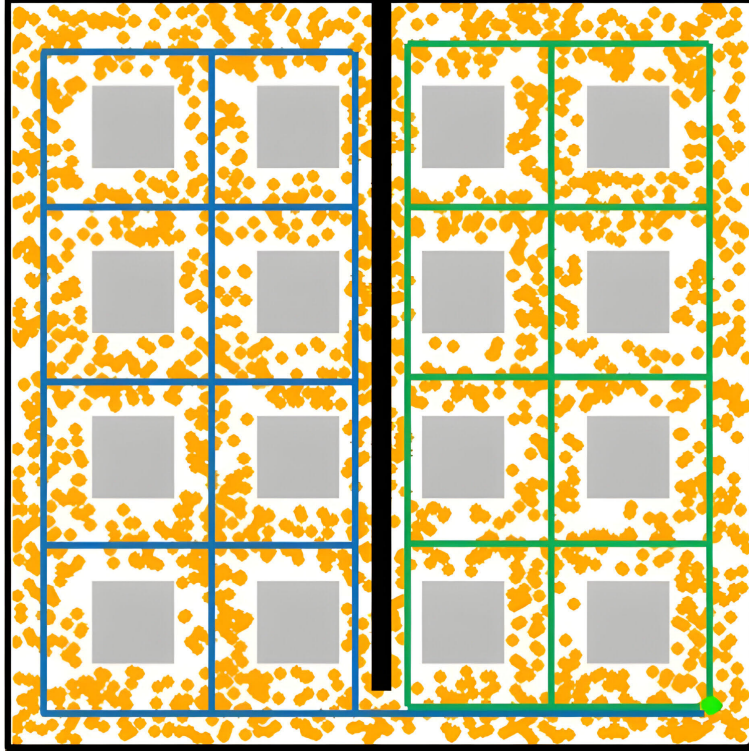


Figure 3.21: Main paths over figure 3.20

- A path is completed when it does not have any candidate children,

$$PSN_{child} \subset PSN_{path} \quad (3.4)$$

for  $P_{child} = P_{path} + \{x\}, \forall x \in LNV_{path}$

- A path is active if it is on the open path list. Define an active path set as a set that contains active paths.
- Assume all sample nodes in the  $C_{free}$  are seen at least once after creating a child of a path. Let us say there are  $m$  active paths. Since the exploration of space is complete, the union of paths' seen nodes vectors are equal to the set of total samples,

$$\bigcup_{i=1}^k \{n_i\} = V_{C_{free}} = \bigcup_{i=1}^m PSN_i \quad (3.5)$$

- If any of the active paths,  $P_n \in ActivePaths$ , have a candidate child, that child sees a node from  $V_{C_{free}} - PSN_n$ . A child sees a node from another active path's  $PSN$ .

- Assume one of the active paths,  $P_n$ , has a candidate child which sees some nodes from  $PSN_l$ , seen node vector of  $P_l$ .  $PSN_n$  lacks some nodes of the  $PSN_l$  since a node can see a portion of  $PSN_l$  can increase  $PSN_n$ .
- Paths expand with their last node's visibility set,  $LVN$ . Thus,  $P_n$  is going to expand through to  $LVN_l$ .
  - If  $P_n$  can see the  $P_l$ 's last node, it should expand with that node since  $P_l$ 's last node can see any node in the  $LVN_l$ .
  - If  $P_n$  cannot see the  $P_l$ 's last node, it will expand with a node from  $LVN_l$ . Refer this child as  $P_m$ .
  - Note that  $P_l$  also expands in the space if it is incomplete.
  - If  $P_m$  is not complete, it's going to expand with  $P_l$ 's last node.
    - \* It's guaranteed that  $P_m$  can see  $P_l$ 's last node, since  $P_m$ 's last node is from  $LVN_l$ .
    - \*  $P_m$ 's last node certainly provides connectivity through  $P_l$ 's previous nodes visibility regions or  $P_l$ 's expansion direction.

Due to this fact, if the number of the roadmap nodes is  $U$  when the exploration of space is complete. The minimum number of roadmap nodes is  $U$  if all the last nodes of the active paths can see each other. The maximum number of roadmap nodes is  $U + (AP - 1)$ , where  $AP$  is the number of active paths if any of the active paths' last node cannot see any other roadmap node.  $\square$

Theorem 3.5.6 lets us iterate over roadmap nodes after the exploration of the space is completed. Iterating over roadmap nodes dramatically reduces the number of nodes to iterate for detecting the possible candidates and prunes search space. Thanks to this, the algorithm performs efficiently even in the obstacle-rich configuration spaces. It can solve the figure 3.21 within seconds, while it could not find the solution for hours without theorem 3.5.6.

One last method, applied to boost the algorithm's performance, should be mentioned. The algorithm tries to find every alternative sub-optimal path while exploring and connecting the configuration space. Many paths share some common nodes, and they

|             |             |             |             |             |             |             |             |             |
|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
| <i>None</i> | 1           | 1           | 0           | 0           | 0           | 0           | 0           | 1           |
| 1           | <i>None</i> | <i>None</i> | 1           | 1           | 0           | 0           | 0           | <i>None</i> |
| 1           | <i>None</i> | <i>None</i> | 1           | 0           | 1           | 1           | 0           | <i>None</i> |
| 0           | 1           | 1           | <i>None</i> | 1           | 0           | 0           | 1           | 1           |
| 0           | 1           | 0           | 1           | <i>None</i> | 0           | 0           | 0           | 1           |
| 0           | 0           | 1           | 0           | 0           | <i>None</i> | 1           | 0           | 0           |
| 0           | 0           | 1           | 0           | 0           | 0           | <i>None</i> | 1           | 1           |
| 0           | 0           | 0           | 1           | 0           | 0           | 1           | <i>None</i> | 0           |
| 1           | <i>None</i> | <i>None</i> | 1           | 1           | 0           | 1           | 0           | 0           |

Figure 3.22: Adjacency matrix of figure 3.17

try to expand from that node through configuration space. During this step, a path might call a collision check operation for **Node X** and **Node Y**. This information was then calculated but not saved. Thus, any path that expands with **Node X** and does not include **Node Y** in its seen node vector is going to call the collision check operation for **Node X** and **Node Y** again. This behavior is highly unpleasing since it is desired to decrease the number of collision checks. Thus, an adjacency matrix is defined to keep track of the results of the collision check operations. Figure 3.17 shows the adjacency matrix of figure 3.22. The row index represents the path's initial node, and the column index represents the path's final node. **None** represent collision check is not performed, **0** means there is no feasible path between nodes and **1** represent feasible path between nodes. For example,  $M[1, 3] = 0$  means there is not a feasible path from **Node 1** to **Node 3**. Diagonal is filled with **None** since checking the node itself is unnecessary. Moreover, there are some extra **Nones** in the adjacency matrix since the algorithm did not need to check those connections.

### 3.6 Building Regions

The algorithm can efficiently find alternative complete paths that cover the configuration space. A result of figure 3.11 is shown in figure 3.23 and figure 3.24. In figure 3.23 roadmap nodes, and in figure 3.24, complete paths are shown. The total

number of roadmap nodes is 198, and the total number of complete paths is 1132. However, the algorithm finds some extra nodes and paths. These nodes and paths are shown at 3.25. Yellow regions contain nodes extremely close to each other, and red regions show almost similar paths. We defined the dominated nodes at lemma 3.5.2. However, those extra red and yellow regions are caused by nodes that see at least one node; their neighbor nodes could not see that, even though they are incredibly close. In order to eliminate those extra paths and nodes, we will define a similar term, like dominated/dominating nodes.

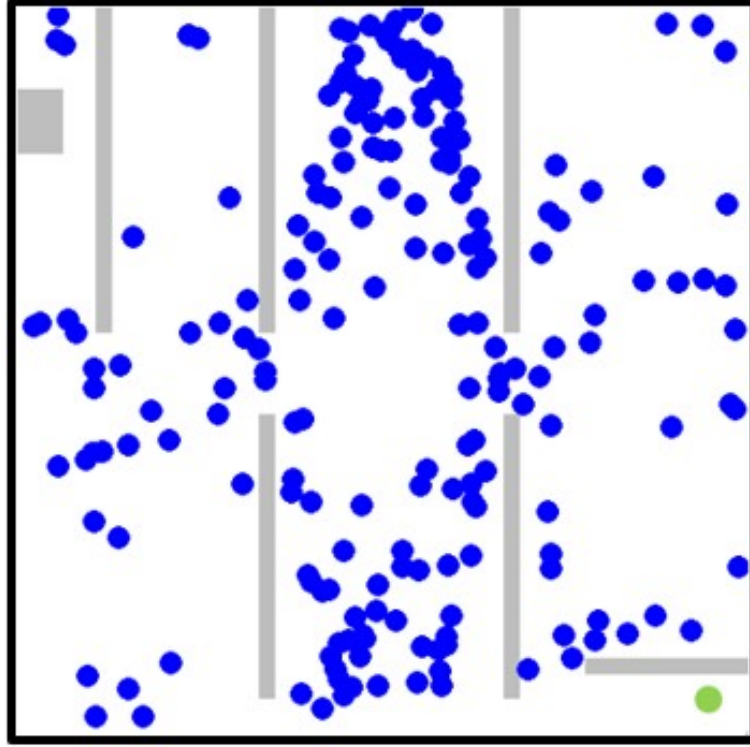


Figure 3.23: Roadmap nodes, 198 nodes, over 3.11

**Lemma 3.6.1.** *If a candidate path,  $P_{candidate}$ , can not dominate a previously found candidate path,  $P_{other}$ , but they are closer than a threshold, **brotherhood threshold**,  $P_{other}$  discarded from the candidate list and last node of the  $P_{other}$  is inserted into last node of the  $P_{candidate}$  as a **brother node**. Note that the main and brother nodes must see each other.*

A brother node cannot create a path, and a brother node is not shown in the roadmap. However, the brother node sees at least one node that the main node cannot see. There

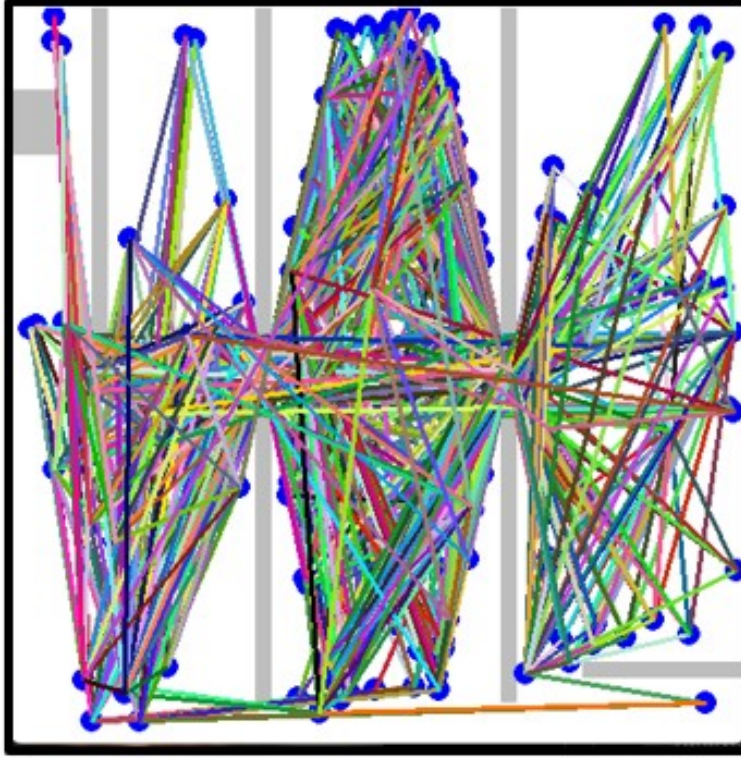


Figure 3.24: Complete paths, 1132 paths, over figure 3.11

is a not feasible path information saved inside the main node. Difference between a brother node and a dominated node can be expressed as,

- **Brother node:** Contains at least one path, which is not feasible for the main node.
- **Dominated node:** Contains a possibly more optimal path than the main node.

The brotherhood threshold is a distance that eliminates candidate paths, that their last nodes are closer than a distance. It can be selected with various approaches. In this study, we used two times the average distance between sample nodes as the brotherhood threshold. Since we want every node to check for nodes around itself, if the value is lower than the average distance between sample nodes, nodes cannot check most of their neighbors.

Results obtained with lemma 3.6.1 are shown in figure 3.26 and figure 3.27. The total number of nodes decreased by %46 and became 127 nodes. The total number of paths

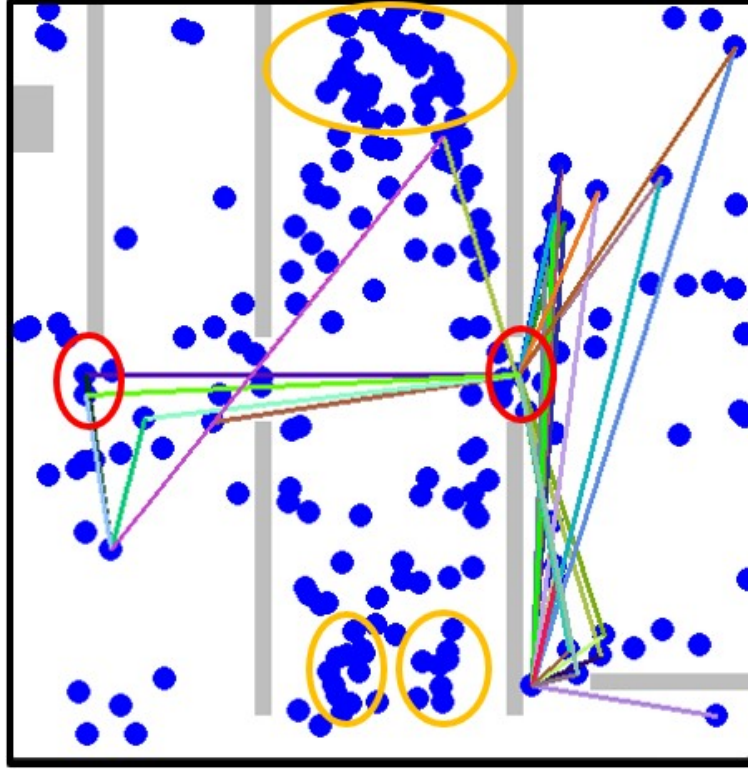


Figure 3.25: Extra found nodes and paths by the algorithm

decreased by %49 and became 580 paths.

The final results are satisfying since the algorithm catches all the connectivity of a configuration space sampled with approximately 3000 nodes with only 127 nodes. Found paths give all alternative sub-optimal paths. However, there are still some improvements that can be made. Especially in the discrete optimization part, we will calculate the costs of the paths, and we want to find similar paths since switching between them for different targets will decrease our total cost. In order to achieve this, we are going to define **regions**.

**Lemma 3.6.2. Region,  $R$ ,** is a set of nodes whose paths consist of at most one element of the region, and every node in the region can see other nodes.

Regions are categories of the nodes. Since every path contains at most one node from a region, there is no need to take more than one state from any region to reach the goal state or place more than one drone in a region to hit a target in the case study problem. Regions can be thought of as cities of a country. Assuming we have a new

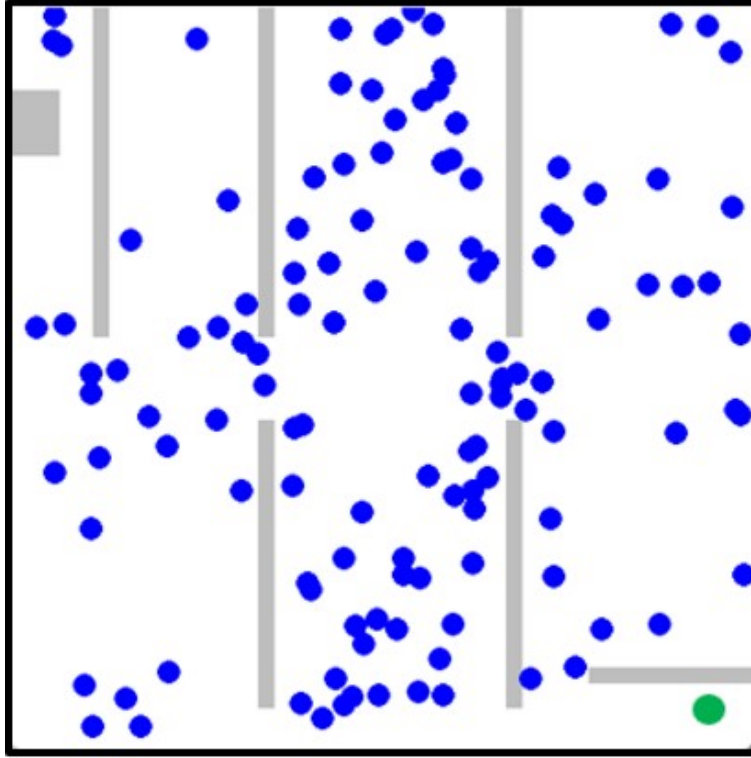


Figure 3.26: Roadmap nodes, 127 nodes, over 3.11 with brotherhood

land and want to build a country over the land, then regions can be considered optimal positions of the cities to provide total connectivity over the country. Moreover, for a high-dimensional robot motion planning problem, regions can be thought of as the primary poses of the robot to be able to perform every move the robot can perform. Like the brotherhood concept, we will define a threshold, referenced as **region threshold**. **Region threshold**, the maximum distance a node checks its environment in the configuration space to include its neighbors to its region. A higher region threshold creates fewer regions than a lower threshold, but non-similar nodes might be categorized into the same region. A lower region threshold creates more regions and might create unnecessary regions. However, the minimum value should be equal to the brotherhood threshold to create regions successfully. Clustering methods might be used to define the ideal region threshold for different problems. Created regions over figure 3.11 are shown in figure 3.28. The total number of created regions is 69, with the region threshold equal to two times the brotherhood threshold.

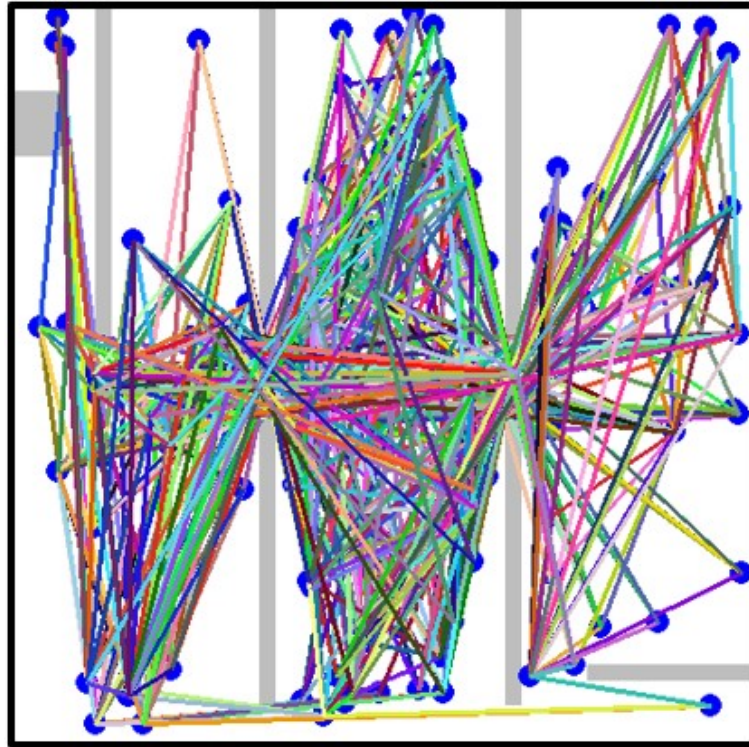


Figure 3.27: Complete paths, 580 paths, over figure 3.11 with brotherhood

### 3.7 Finding Solution to a Goal Node

Goal nodes can be inserted in various ways. One alternative way to insert a goal node is by checking the visibility between the goal node and roadmap nodes. For every roadmap node that can see the goal node, the roadmap node's paths are tried to expand through the goal node. Another approach would be iterating over paths. From starting the root path, we could check if a path's last node can see the goal node. If the last node of the path can see the goal node, the path expands with the goal node and is saved as a solution. If the last node of the path cannot see the goal node, the path's children are added to the iteration list. Using the node approach requires checking whether a path is sub-optimal. However, the path approach might check unnecessary paths. For goal nodes that require low state changes, the number of nodes in sub-optimal paths benefits from the path approach. In this study, we are going to continue with the path approach.

Example goal node insertion and solution paths are shown in the figure 3.29. The

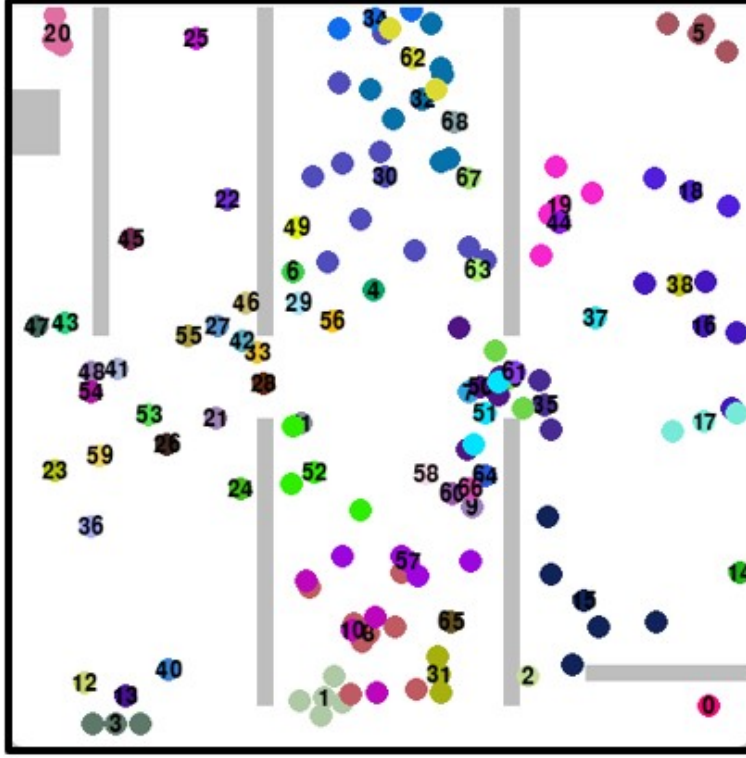
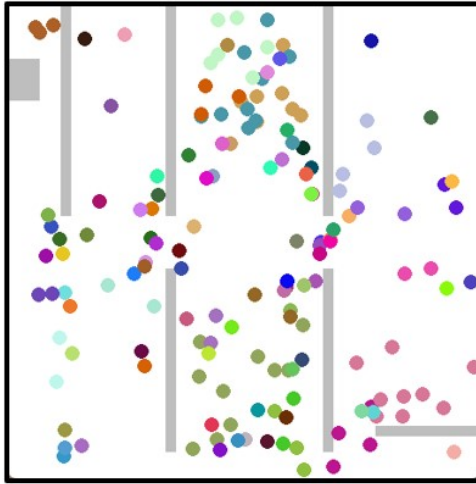


Figure 3.28: Regions, 69 region, over figure 3.11

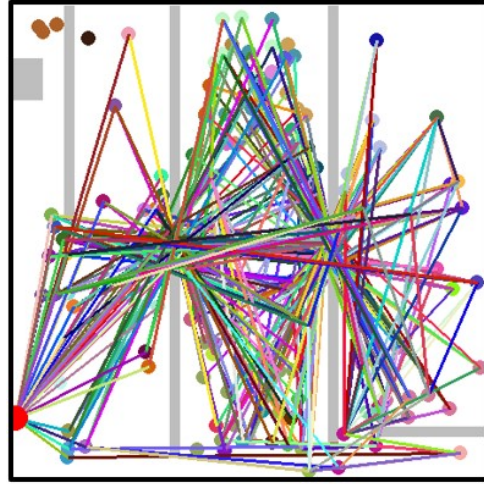
created roadmap is shown in figure 3.29a, and different goal nodes and their found alternative solutions are shown in figure 3.29b, 3.29c and 3.29d. Now that we have the alternative sub-optimal solutions to the node, we can apply the discrete optimization to solve the DOPs.

### 3.8 Region Orders

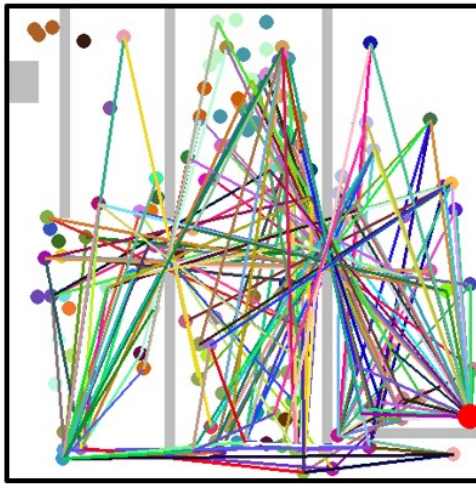
Roadmap nodes are successfully classified as regions, and now, regions will be used to calculate the costs of the paths. In order to achieve this, for every solution path that leads to  $x_g$  from  $x_i$ , a **region order** is defined. **Region order** is a list that contains region numbers for intermediate nodes of the paths. For example, if a solution path is found as  $P_{solution} = \{Node_0, Node_3, Node_{45}, Node_{23}, Node_{goal}\}$ , then region order is defined as,  $RO = \{Region_{Node_3}, Region_{Node_{45}}, Region_{Node_{23}}\}$ . Region order is important for the DOPs' solution since we will calculate the region cost according to the current state of the world. Then we will decide the optimal solution.



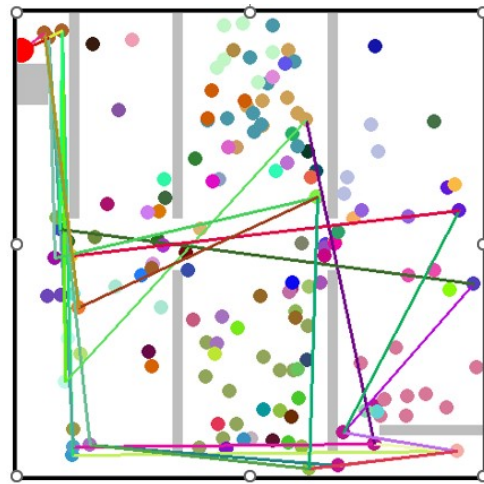
(a) Regions, 69 region, over figure 3.11



(b) First goal node



(c) Second goal node



(d) Third goal node

Figure 3.29: Different goals, big red nodes, and their solutions



## CHAPTER 4

### FORMULATION OF THE CASE STUDY

In this chapter, the problem formulation of the case study is shared.

#### 4.1 Formulation

Let  $C$  be defined as a continuous state space. Define the region occupied by obstacles in  $C$  as  $C_{obs}$ . Then, free space can be defined as

$$C_{free} = C / C_{obs} \quad (4.1)$$

Define the position of the laser source as  $x_0$  where

$$x_0 \in C_{free} \quad (4.2)$$

Assume there are  $n$  drones available, and the positions of the drones are defined as

$$d_i \in C_{free} \quad for \quad i = 1, 2, \dots, n \quad (4.3)$$

Assume there are  $m$  targets are defined, and the positions of the targets are defined as

$$t_i \in C_{free} \quad for \quad i = 1, 2, \dots, m \quad (4.4)$$

Define a path,  $\tau_m$ , a set of positions, for a target,  $t_m$ , if  $\tau(x_0, t_m) \in C_{free}$  where  $\tau(x_0) = x_0$  and  $\tau(t_m) = t_m$ .

**Constrain 1** A path,  $\tau$ , can contain as many elements as  $n+2$  where  $n$  is the number of the drones.

$$n(\tau) \leq m \quad (4.5)$$

The problem's solution,  $\tau_S$ , is a set of paths that includes feasible paths for every target.

#### 4.1.1 Assumptions

In order to solve the problem the problem effectively, some assumptions are made. Assumptions can be listed as,

1. The number of drones in the system is limited.
2. A drone can reflect incoming laser at any angle.

#### 4.2 Applying Discrete Optimization to Find Optimal Order of Targets

Roadmap nodes are successfully classified as regions, and the region order for every path can be found. In the selected case study problem, we will calculate the costs of every region for every drone, and then we will decide the total cost of a path, based on the regions in its region order.

Let us model the discrete optimization problem for the case study,

- Define a drone with the following attributes,
  - Position: Active position of the drone
  - Battery: Drone's resource to perform flight
  - Mission End Time: The time that the drone is available after performing its mission
- Define an objective function,

- Minimum amount of time to hit all targets
- Define constraints,
  - A drone must always have enough battery to return to  $x_i$  for recharge
  - In order to eliminate a target, the laser must be exposed for an interval of time
  - Only a target can be exposed to laser at a time
  - Mission is completed when all targets are eliminated

We will use Large Neighborhood Search, LNS, to perform discrete optimization. LNS has been chosen for its outperforming performance in recent years over other optimization methods. Moreover, by definition, LNS is highly suitable for adapting the appearance of new targets on the map. LNS can continuously improve the solution and can optimize any sub-part of the problem. For example, LNS can optimize the sequence of the third, fourth, and fifth targets. LNS can change their configurations to hit.

The main flow of the LNS can be explained as,

1. Find a feasible solution with a greedy approach
  - (a) For every region, calculate the drone costs to arrive in that region based on,
    - Arrival time of the drone to region
    - Drone's battery
  - (b) For each target in the target list, find solution paths' region orders
  - (c) For each region order, make drone-region matching and calculate costs of region order
  - (d) Select the target that has the region order with the lowest cost
    - i. Check paths defined with that region order in the selected target node
    - ii. Select the path with minimum cost
  - (e) Assign drones to their specified regions

- (f) Update battery of the traveled drones
- (g) If a drone needs to be recharged, send it to  $x_i$
- (h) Update the mission end time of drones by considering travel and target-eliminating time.
- (i) Remove selected target from the target list
- (j) If the target list is not empty, return to step  $c$

2. Apply LNS,

- Select a sub-part of the problem
- Optimize the selected part

## CHAPTER 5

### SIMULATION RESULTS AND DISCUSSION

This chapter presents various maps, their dense sampling, roadmap nodes, complete paths, and solutions for various targets for each map. After that, a section is devoted to the discussion of the obtained results.

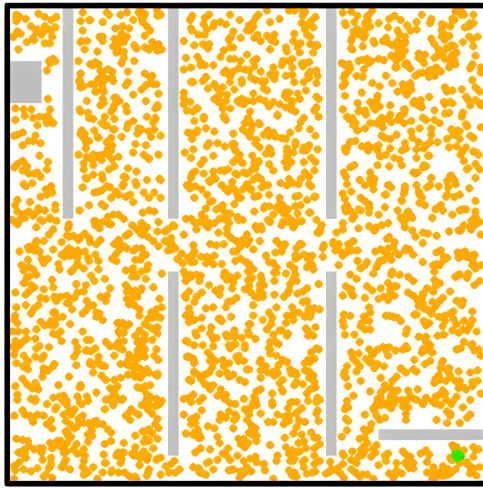
#### 5.1 Simulation Results

##### 5.1.1 Results for Map 1

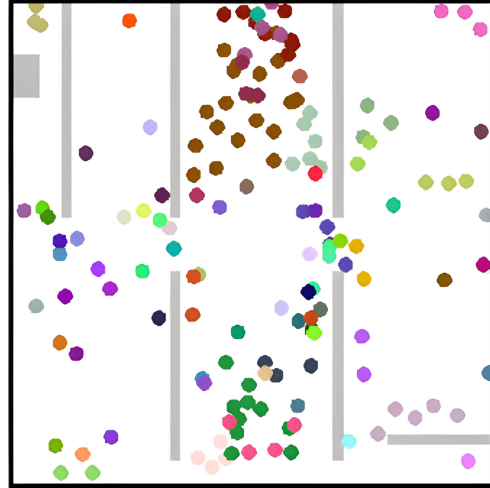
Map 1 is selected as the map shared in figure 5.1 since we showed the design of the algorithm with that map. Map 1 is sampled with 2621 samples. The map's height and width are 450 units. The brotherhood threshold is selected as 18 units, and the region threshold is selected as 36 units.

##### 5.1.1.1 Map 1, Scenario 1

Figure 5.2 shows four goals selected as scenario 1 for the first map. Alternative paths to eliminate the targets depend on the number of drones. Figure 5.3 shows four goals and their possible optimal solutions for two available drones. Figure 5.4 shows four goals and their possible optimal solutions for five available drones. Increasing the number of available drones in the scenario increases the alternative paths. Figure 5.5 shows the case study solution. The case study solution is obtained for two drones without any battery considerations. However, to eliminate a target, the laser must be exposed for 100 units of time. The blue circle labeled 1 in the figure 5.5 represents



(a) Map 1 sampled with 2621 nodes



(b) Regions, 79 region, over 5.1a

Figure 5.1: Map 1 and created regions

the first drone, and the green circle labeled 2 represents the second drone. In 5.5, it is shown that algorithm sequenced the goals as figure 5.3a, figure 5.3b, figure 5.3c and figure 5.3d. The algorithm selected the first target to hit with two drones, then decided to hit the second target with the first drone, meanwhile moved the second drone to hit the third target and used the third target's eliminating time to move the first drone to hit the fourth target. The algorithm tried to minimize the total time to hit all targets by positioning a drone while another was eliminating a target.

#### 5.1.1.2 Map 1, Scenario 2

Figure 5.6 shows two goals selected as scenario 2 for the first map. Figure 5.7 shows the case study solution. Like the solution of scenario 1, to eliminate a target, the laser must be exposed for 100 units of time. The blue circle labeled 1 in the figure 5.7 represents the first drone, and the green circle labeled 2 represents the second drone. In 5.7, it is shown that the algorithm sequenced the goals as figure 5.7a and figure 5.7b. The algorithm selected the first target to hit with two drones, then decided to hit the second target by only moving the second drone, even though repositioning the first drone can also hit the second target. The decision is based on minimizing the required time to eliminate targets.

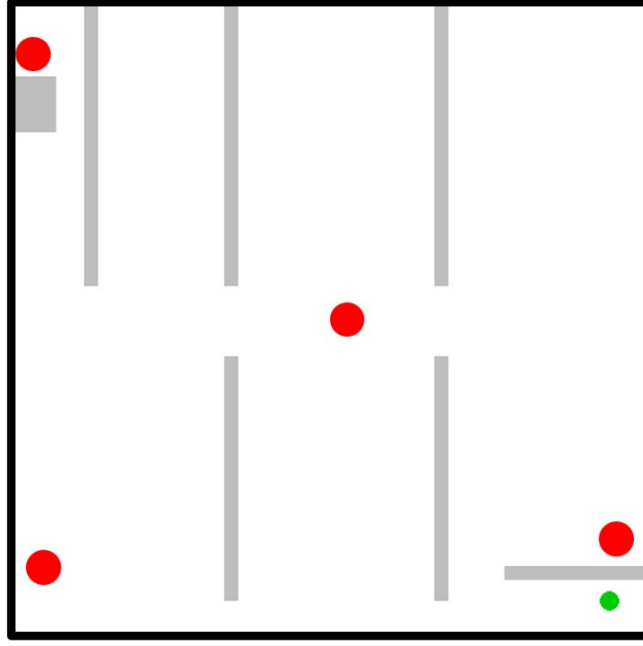


Figure 5.2: Map 1, Scenario 1

### 5.1.2 Results for Map 2

Map 2 is selected as the map shared in figure 5.8 since it is one of the most challenging cases for the proposed sampling-based planning method. Map 2 is sampled with 2021 samples. The map's height and width are 450 units. The brotherhood threshold is selected as 18 units, and the region threshold is selected as 36 units.

Figure 5.9 shows four goals selected as the scenario for the second map. Alternative paths to eliminate the targets depend on the number of drones. Figure 5.10 shows four goals and their possible optimal solutions for two available drones. Figure 5.11 shows four goals and their possible optimal solutions for three available drones. Increasing the number of available drones in the scenario increases the alternative paths. Figure 5.12 shows the case study solution. The case study solution is obtained for three drones without any battery considerations. However, to eliminate a target, the laser must be exposed for 100 units of time. The blue circle labeled 1 in figure 5.12 represents the first drone, the green circle labeled 2 represents the second drone, and the orange circle labeled 3 represents the third drone. In 5.12, it is shown that algorithm sequenced the goals as figure 5.11a, figure 5.11c, figure 5.11b and figure 5.11d. The

algorithm selected the first drone to hit the first and third targets while selecting the second and third drones to hit the second and last targets. It is the same approach as the results of the first map. The algorithm used the targets' elimination time to position the available drones to hit the next target. However, using three drones reduces the required time to eliminate all targets since no time would be gained from having the second and third drones move for the following targets while the first and third targets were hit if the third drone was unavailable.

## 5.2 Discussion

This section presents comments about the results shared in section 5.1. It is shown that the proposed sampling-based planning method can find alternative paths to any goal defined in the space. In simulations presented in section 5.1, the specific orientations' cost is evaluated based on how fast drones can make that formation. Based on the orientations' cost, the planner minimized the required time to hit all targets. This cost function can be updated to include other environmental criteria. For example, sub-parts of the space can be evaluated for their safety, and orientations can inherit the costs of their regions. Moreover, the case study's primary objective might be to minimize the drones' energy costs or agents might not be equal and have different costs.

In the scenarios presented in the section 5.1, battery condition is relaxed, which means drones have enough battery to complete their mission and return safely. Battery condition is relaxed mainly because strict battery condition makes planner favor the orientations that offer the shortest paths to goals and fast recharging. Relaxed battery condition makes planner favor the orientations that can be achieved during the targets' elimination time by other drones. Moreover, the targets' elimination time and the number of drones are important decision variables for the planner.

In the scenarios presented in the section 5.1, battery condition is relaxed, which means drones have enough battery to complete their mission and return safely. Battery condition is relaxed mainly because of easy understanding of the method. Relaxed battery condition makes planner favor the orientations that can be achieved during the targets'

elimination time by other drones. However, strict battery condition makes planner favor the orientations that offer fast recharging, and this condition starts to make a difference for a high number of drones with a high number of targets. Moreover, the targets' elimination time and the number of drones are important decision variables for the planner. Targets' elimination time is essential since the algorithm can use that time to reposition other drones to hit the following targets. Shorter targets' elimination time requires similar paths to hit sequential targets since the planner would not favor losing time to reposition the drones. The number of drones is essential since it offers more possibilities to the planner.

However, the main focus of this thesis is not tuning a heuristic cost function to improve the planner's performance in the selected case study. The main focus is providing a sampling-based planning method that transforms highly complex search spaces into simplified search spaces to navigate the complex decision-making process of the DOPs. To successfully simplify a search space subject to the dynamic objective function, alternative paths that lead to optimal solutions for any distinct instances of the space should be identified. The appendix A shows that sampling-based planning methods in the literature cannot achieve this. However, section 5.1 shows that the proposed sampling-based planning method can successfully simplify the search space for complex problems. This achievement also shows that the proposed method can contribute to the solution of complex problems in many fields, such as robotics, operational research, and economics related to DOPs. For example, a decision to select a motion to achieve a task for a humanoid robot highly depends on the current conditions of the robot and the environment. Jumping over an obstacle to reach a target position might be cheaper than traveling a long distance to reach the target in normal conditions. However, jumping can be extremely costly if the robot carries a load or its leg is damaged. In both cases, knowing the alternative solutions is the key to finding the optimal decision.

The proposed sampling-based planning method successfully gives the alternative paths for the case study shared in the section 5.1. The primary purpose was to use the outputs of the sampling-based method to optimize the solution by using the Large Neighborhood Search. However, the greedy approach for the selected case study offers outstanding solutions once the alternative paths can be evaluated based on the

drones' current positions. Thus, applied LNS could not improve the initial solution. There might be numerous reasons for this. One would be the poor implementation of the LNS since LNS is a meta-heuristic method, and its performance highly depends on the problem model and defined parameters. Defined parameters for the LNS are the neighborhood definitions. In this study, we defined multiple neighborhoods as reordering targets, switching workloads between drones, and selecting different orientations to eliminate the targets. Another reason would be that the problem is simplified in a way that a complex, high-end problem-solving technique like LNS is unnecessary to solve the problem.

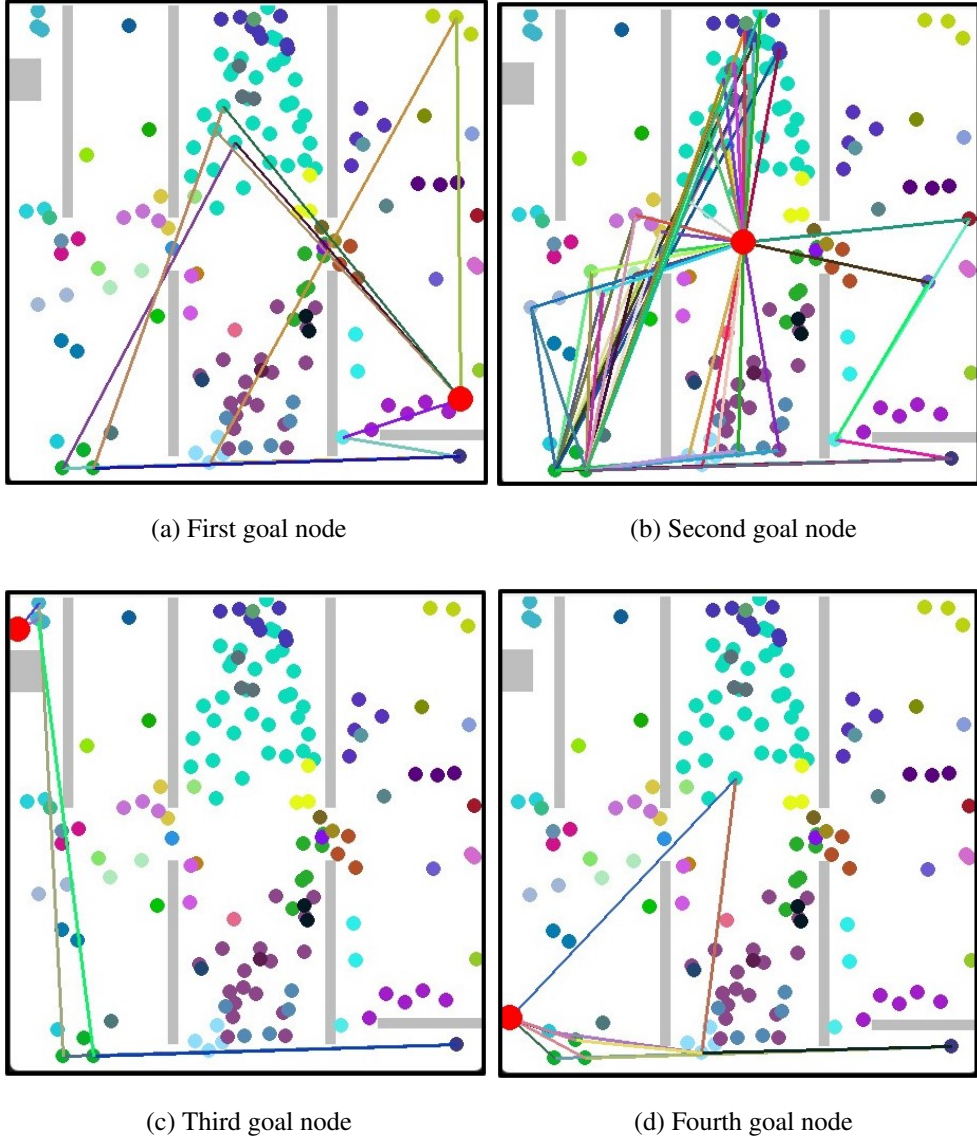


Figure 5.3: Different goals, big red nodes, and their possible solutions for map 1, scenario 1 with two drones

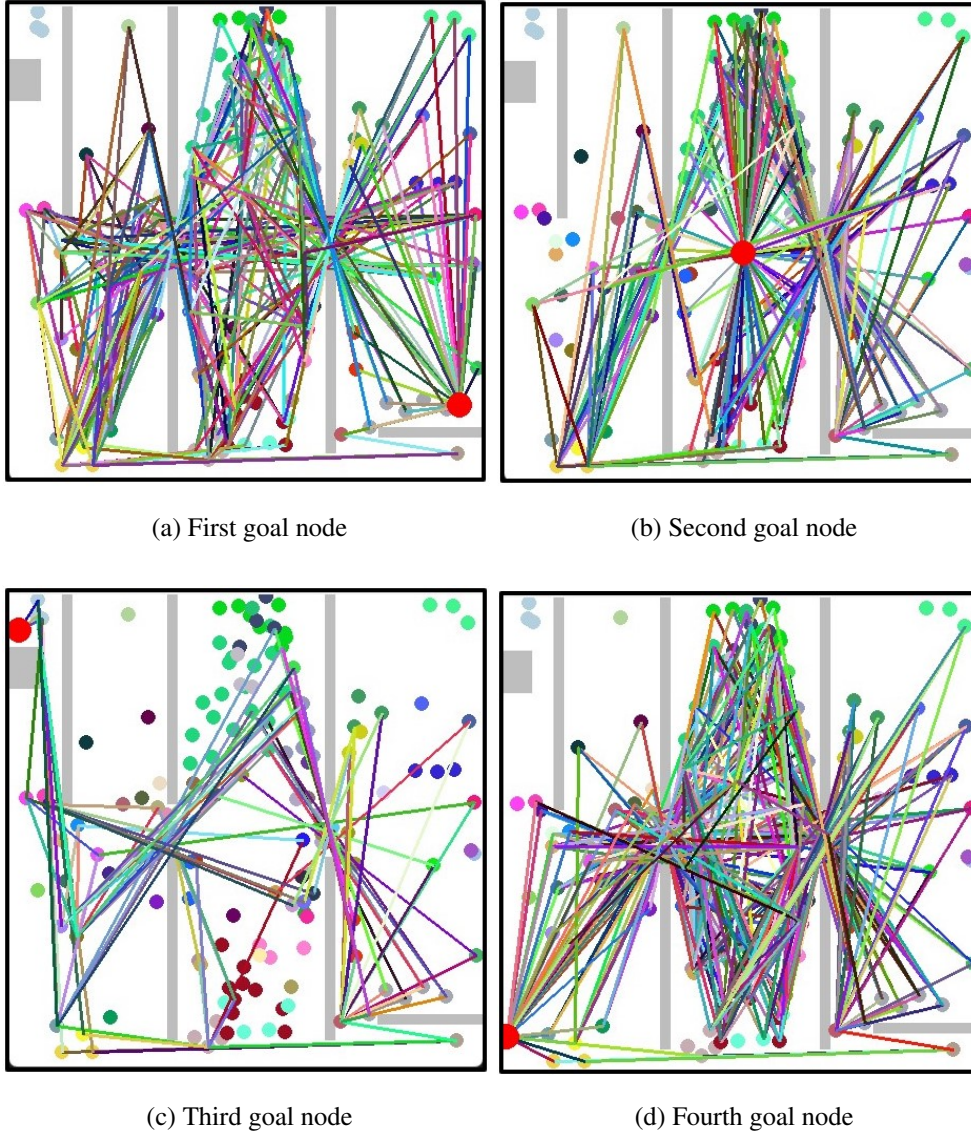


Figure 5.4: Different goals, big red nodes, and their possible solutions for map 1, scenario 1 with five drones

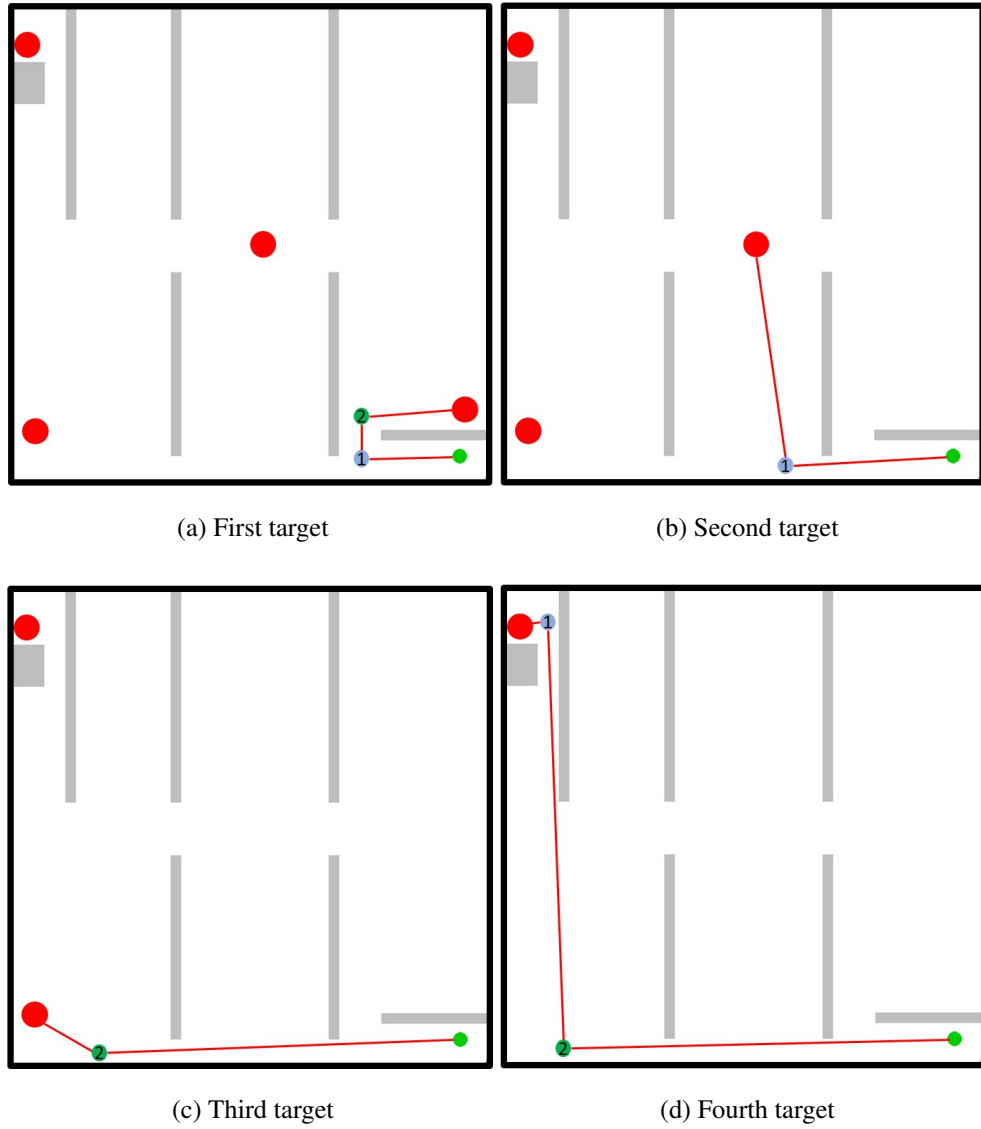


Figure 5.5: Solution of the case study for map 1 for scenario 1 with two drones

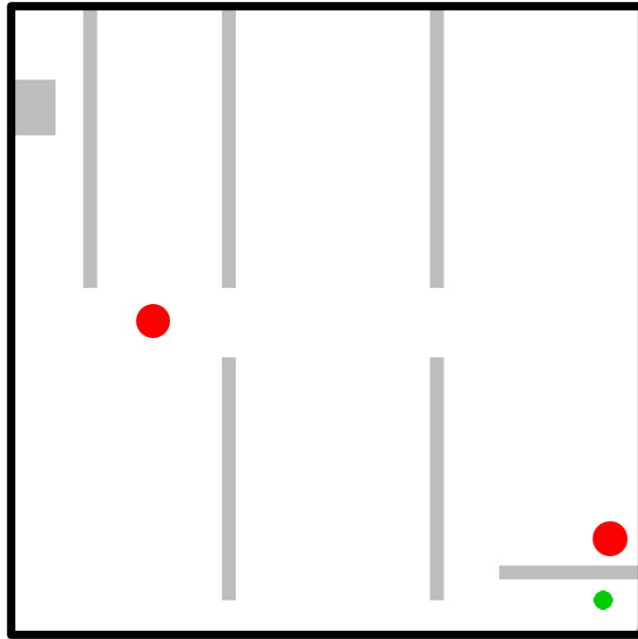


Figure 5.6: Map 1, Scenario 2

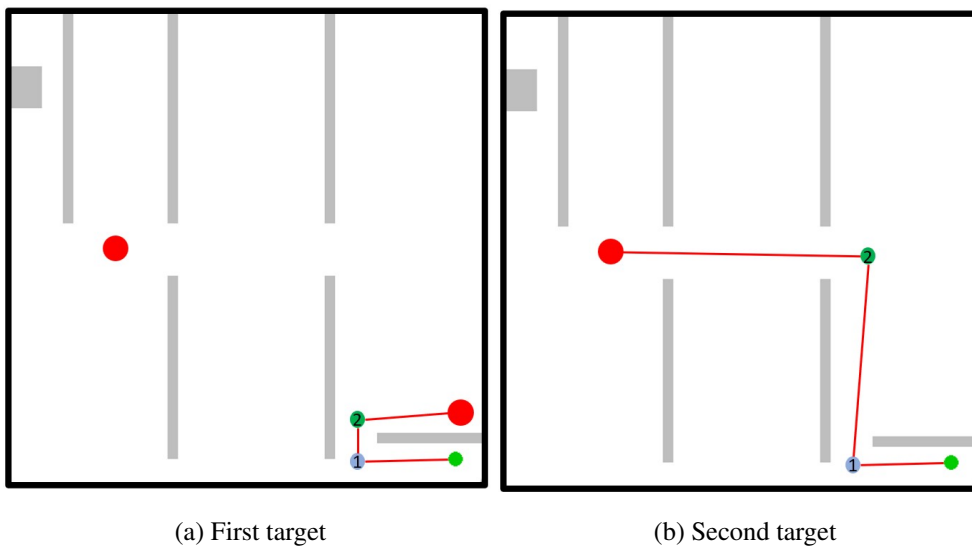
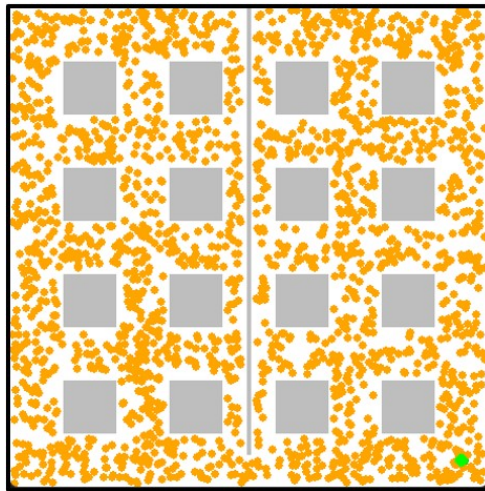
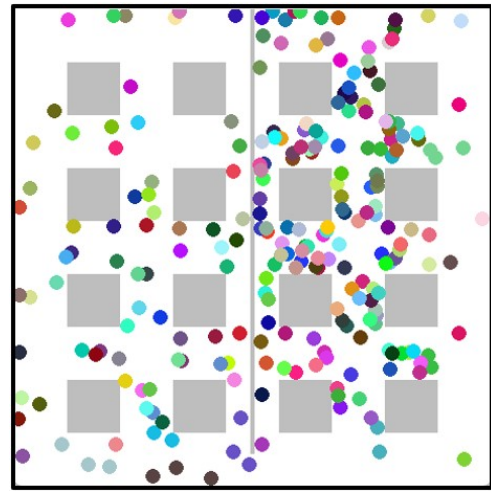


Figure 5.7: Solution of the case study for map 1, scenario 2 with two drones



(a) Map 2 sampled with 2021 nodes



(b) Regions, 231 region, over 5.8a

Figure 5.8: Map 2 and created regions

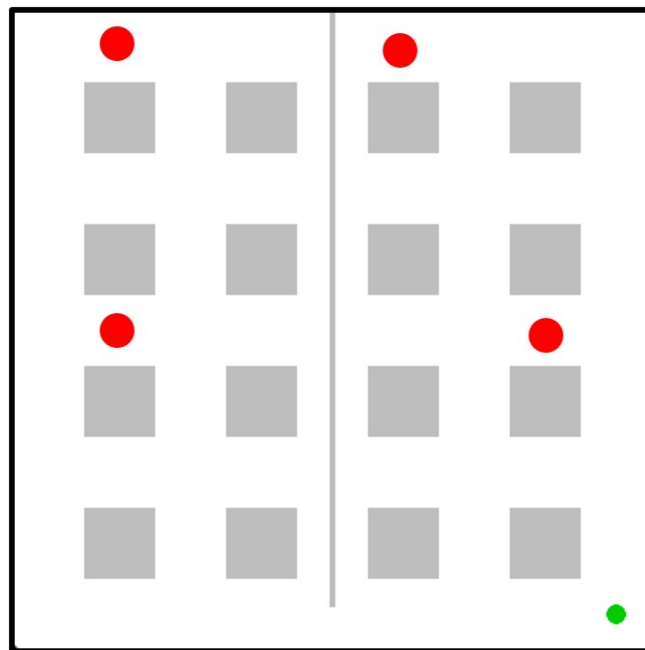


Figure 5.9: Map 2, scenario

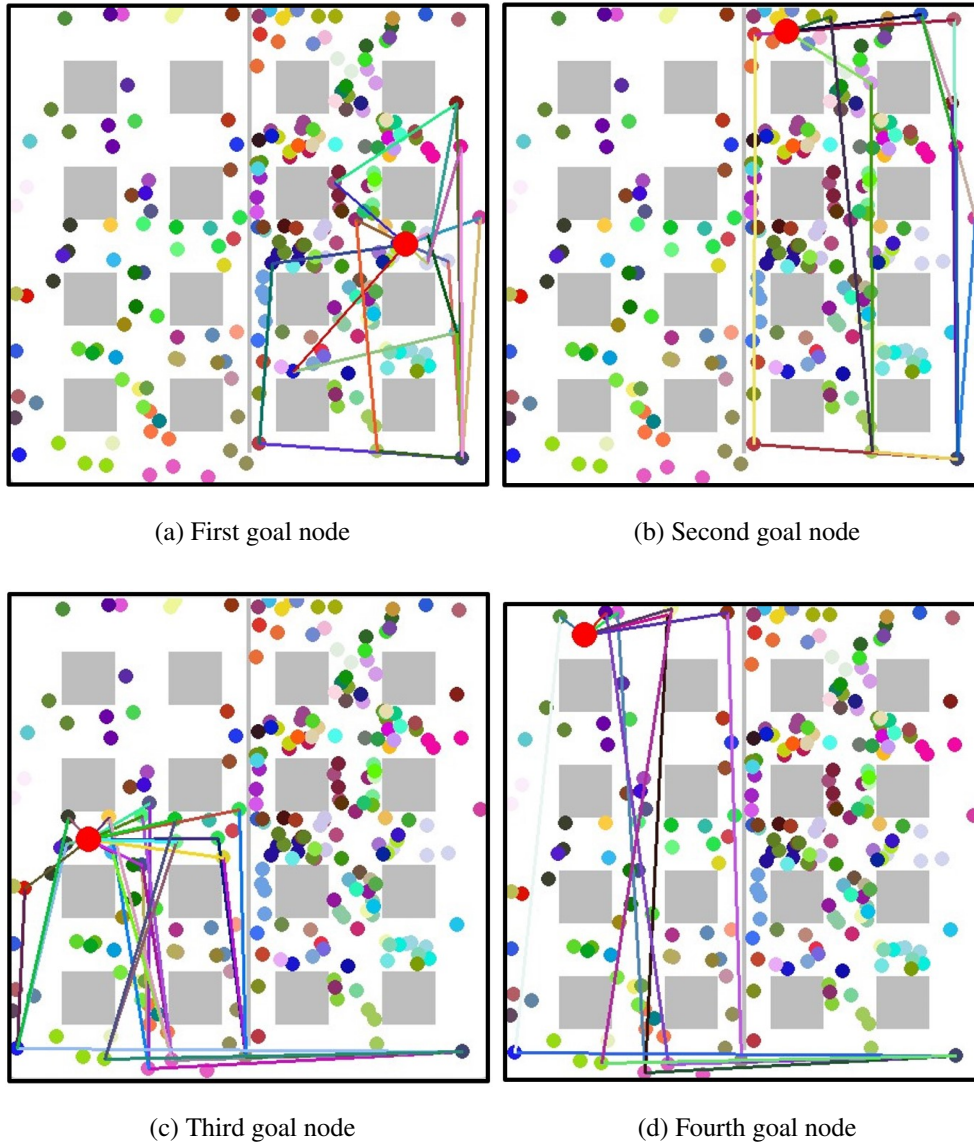


Figure 5.10: Different goals, big red nodes, and their possible solutions for map 2's scenario with two drones

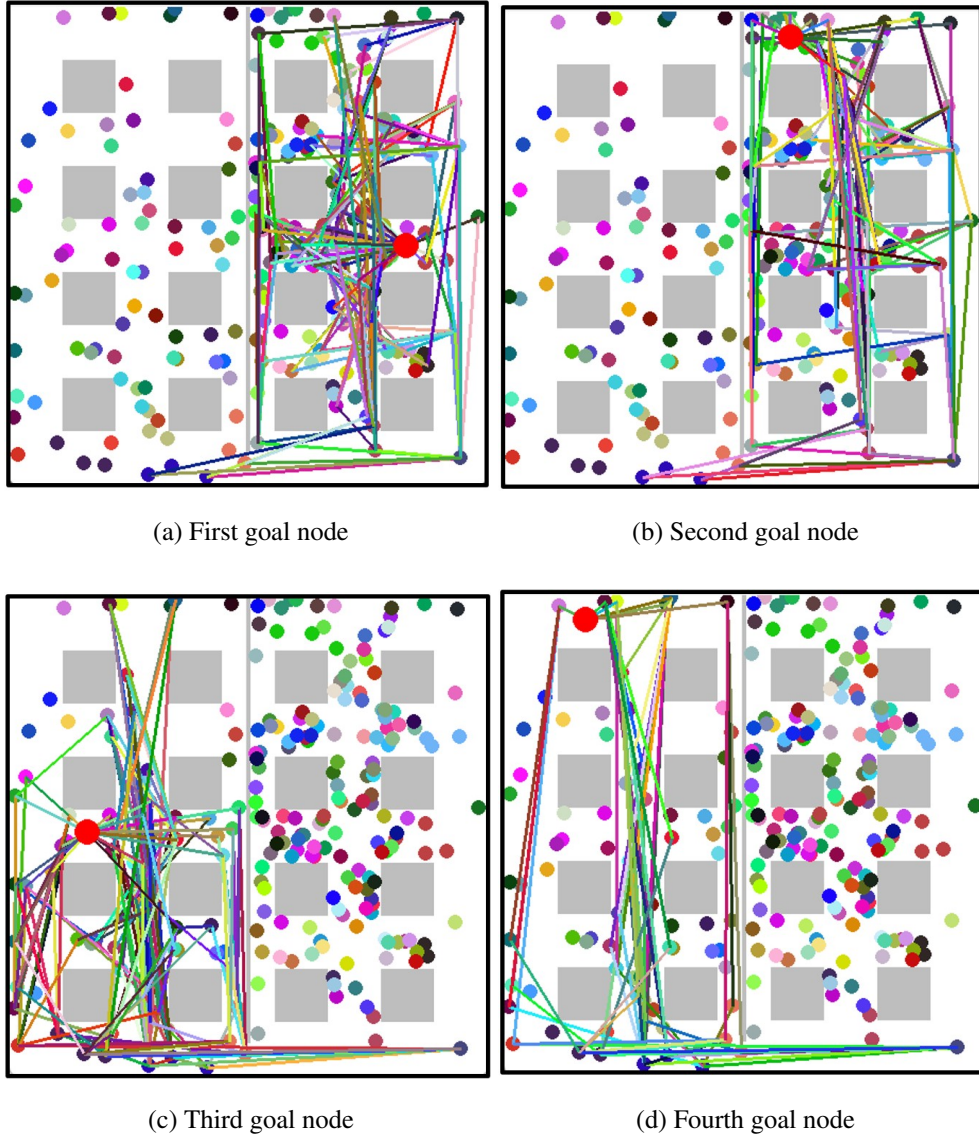


Figure 5.11: Different goals, big red nodes, and their possible solutions for map 2's scenario with three drones

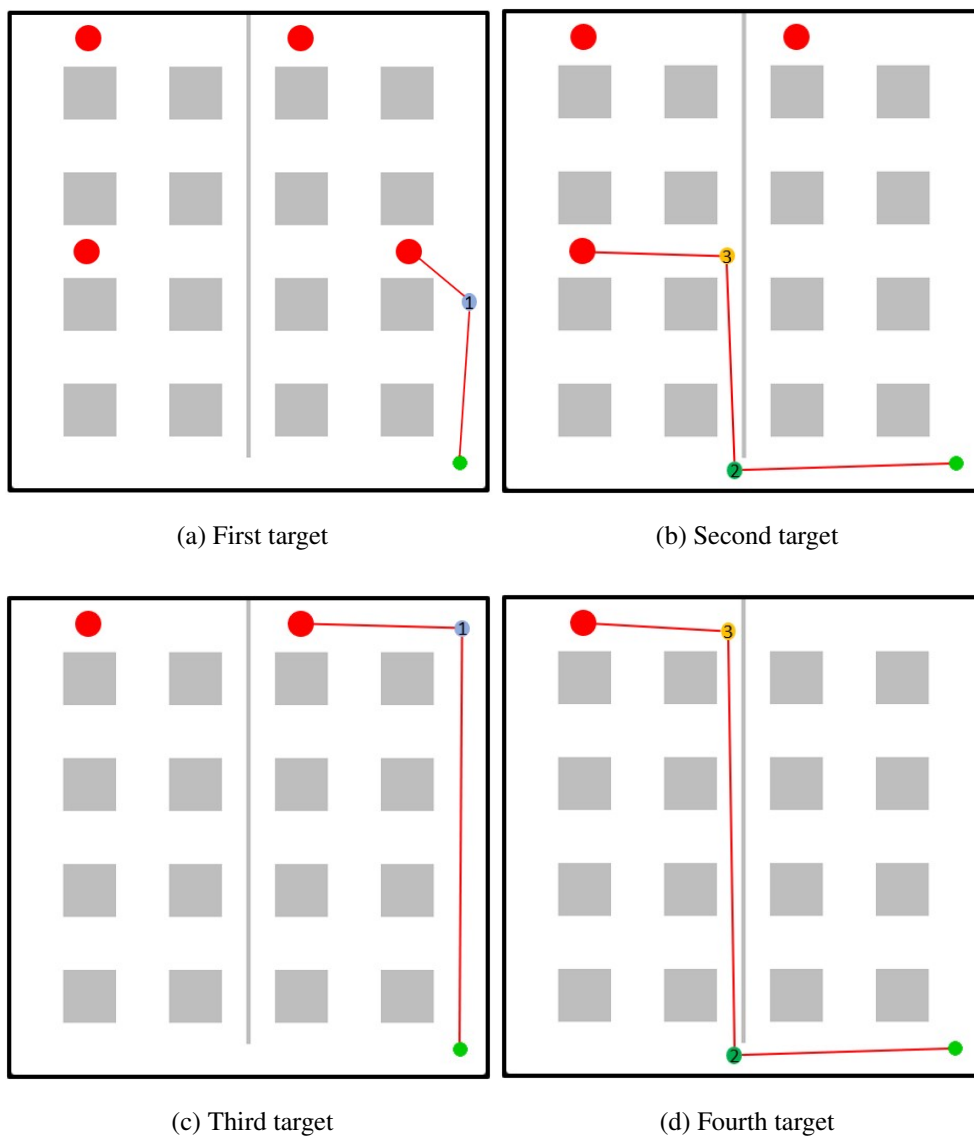


Figure 5.12: Solution of the case study for map 2's scenario with three drones

## CHAPTER 6

### CONCLUSION

This thesis study studies a sampling-based planning technique for resolving dynamic optimization issues with diverse goals and objective functions. The primary idea of the studied planning method focuses on utilizing the efficient exploration capabilities of sampling-based planning to handle the complicated decision-making processes that come with dynamic optimization problems. The studied method can be adapted to various other planning problems since it is generic, like other sampling-based planning methods.

A new search approach, Possible Optimal Paths Search, POPS, is proposed in this thesis as the fundamentals of the proposed sampling-based planning method. POPS is a search method to find any path from a single source that can be optimal for a given set of weights to edges or nodes in a graph. The main aim of POPS is to find alternative paths to any goal in a graph with dynamic weights, which makes POPS suitable for solving dynamic optimization problems on graphs. POPS requires a graph to hold the triangle inequality, which is required for all of the heuristic search methods to be efficient. However, POPS can overcome this requirement of triangle inequality by eliminating triangular connections from the graph by defining imaginary nodes. The output of the POPS is a set of paths from a single source to all nodes in a graph. Every found path is optimal for at least one node in the graph for a given set of weights. Thanks to this, POPS can efficiently find optimal paths after the dynamic behavior of the weights. Moreover, POPS can be used to find pareto-optimal set in multi-objective problems, since objective functions can be applied to possible paths to calculate costs efficiently.

The proposed sampling-based planning method creates a roadmap containing alter-

native paths that might be optimal for a goal state in an instance of the space. The method is designed to minimize the search costs to find the optimal solution of any defined goal in the space for any state of the space. Thus, it encompasses the optimal solution for dynamic objective functions and goal configurations. These features of the proposed sampling-based planning method are achieved using POPS's main ideas to build the paths in space. Built paths and created roadmap provide a simplified search space for a highly complex problem and any discrete optimization algorithm can be used to find optimal solutions for DOPs. However, the first simulation method showed poor or impractical performance, especially in complex obstacle-rich spaces. Exploration and connectivity are defined to improve the method's performance in complex and obstacle-rich environments. Concepts of dominated/dominating nodes, brotherhood, and regions are defined to improve the overall performance. Moreover, the method is designed for batch sampling, which is suitable for parallel computing and will boost the method's performance.

In this thesis, a case study is selected to apply the proposed methods. The case study is selected in a way that consists of a dynamic objective function and goals. The case study defines delivering a laser beam to sequential targets from a laser source by reflecting the beam with drones. In this case study, targets can be hit with different orientations, and the orientations' cost varies with the drones' active positions. Thus, finding the optimal sequence of the targets with the specific orientations is challenging, and the case study consists of dynamic costs and goals. The main objective of the case study is to minimize the total required time to eliminate all targets.

All the simulations and code integration are applied in Python. Pygame is preferred for visualizing the outputs, [22]. Simulation results with different spaces are shared for the selected case study. Results consist of the created paths and roadmap by the proposed sampling-based method. Any discrete optimization algorithm can use the outputs of the proposed method to find the optimal solution. In the scope of this thesis, Large Neighborhood Search is selected as the discrete optimization method. In recent years, LNS has outperformed other optimization methods in assignment and scheduling problems, [20]. However, as mentioned in the section 5.2, LNS could not improve the initial solution found by the greedy approach. Two possible reasons are highlighted for the failure of the LNS. The first one is stated as the poor imple-

mentation of the LNS; since LNS is a meta-heuristic method, its performance highly depends on the problem model and the parameter definitions. The second one is stated as the problem greatly simplified by the proposed sampling-based method; thus, the greedy approach was enough to solve the problem.

In summary, a new search method and sampling-based planning for dynamic optimization problems are studied in this thesis. A case study is selected to apply the proposed methods. The case study is highly complex for the methods in the literature, and it needs to be clarified how to model the problem for optimization methods. In appendix A.1.1, the sampling-based planning methods in the literature cannot provide the optimal alternative paths with dynamic costs and goals. The proposed sampling-based method showed that it could transform highly complex dynamic optimization problems into simplified optimization problems by providing a vastly pruned search space that includes optimal paths for any goal state. The proposed method can be used in many problems, including dynamic costs and goals.



## REFERENCES

- [1] D. Yazdani, R. Cheng, D. Yazdani, J. Branke, Y. Jin, and X. Yao, “A survey of evolutionary continuous dynamic optimization over two decades-part a,” *IEEE Transactions on Evolutionary Computation*, vol. 25, pp. 609–629, 8 2021.
- [2] T. T. Nguyen, S. Yang, and J. Branke, “Evolutionary dynamic optimization: A survey of the state of the art,” *Swarm and Evolutionary Computation*, vol. 6, pp. 1–24, 10 2012.
- [3] H. Fu, P. R. Lewis, B. Sendhoff, K. Tang, and X. Yao, “What are dynamic optimization problems?,” 2014.
- [4] L. E. Kavraki, P. Švestka, J. C. Latombe, and M. H. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, 1996.
- [5] S. M. LaValle, “Rapidly-exploring random trees: A new tool for path planning,” 10 1998.
- [6] S. M. Lavalle, *PLANNING ALGORITHMS*. Cambridge University Press, 2006.
- [7] E. F. Moore, “The shortest path through a maze,” 1959.
- [8] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, 1959.
- [9] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, 1968.
- [10] B. Chazelle, *Approximation and decomposition of shapes*. 1987.
- [11] N. J. Nilsson, *Principles of Artificial Intelligence*. 1987.
- [12] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” vol. 30, 2011.

- [13] L. Janson, E. Schmerling, A. Clark, and M. Pavone, “Fast marching tree: A fast marching sampling-based method for optimal motion planning in many dimensions,” *International Journal of Robotics Research*, vol. 34, 2015.
- [14] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, “Informed rrt: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic,” 2014.
- [15] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, “Batch informed trees (bit): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs,” vol. 2015-June, 2015.
- [16] T. Siméon, J. P. Laumond, and C. Nissoux, “Visibility-based probabilistic roadmaps for motion planning,” *Advanced Robotics*, vol. 14, 2000.
- [17] A. H. Land and A. G. Doig, “An automatic method of solving discrete programming problems,” *Econometrica*, vol. 28, 1960.
- [18] F. Rossi, P. van Beek, and T. Walsh, *Handbook of Constraint Programming (Foundations of Artificial Intelligence)*. 2006.
- [19] G. Sierksma and Y. Zwols, *Linear and integer optimization: Theory and practice, third edition*. 2015.
- [20] M. Gendreau and J.-Y. Potvin, *Handbook of Metaheuristics Third Edition*. 2019.
- [21] P. Shaw, “Using constraint programming and local search methods to solve vehicle routing problems,” vol. 1520, 1998.
- [22] P. D. Team, “Pygame.” <http://pygame.org/>, 2011.

## APPENDIX A

### EVALUATIONS OF THE METHODS IN THE LITERATURE SURVEY

In this chapter, an evaluation, based on the formulation and the constraints of the case study, of the methods mentioned in Chapter 2 is presented.

#### A.1 Method Evaluation

##### A.1.1 Planning Algorithms' Evaluation

In figure A.1, the laser source is illustrated as **green node**, and the goal is illustrated as **blue node**. Three possible paths for hitting the **blue node** are shown with colored arrows. There are infinitely many solutions in a continuous state-space since every infinitesimal movement of the arrows also creates feasible solutions. Thus, deciding the positions of the drones is a challenging task.

In figure A.2, densely sampled, with 3000 samples, state-space is shown. **Green node** represents the position of the laser source. Directly applying a search method, like BFS, DFS, or  $A^*$ , is not possible without determining the neighborhood relationship. Assume we defined the neighborhood relationship as all possible nodes without concern about the space size. Suppose a search method is applied directly as its formal definition. In that case, the search method is not going to find different possible orientations shown in figure A.3 because the visited nodes are not rechecked after they are marked as visited once, and all paths shown in figure A.3 shares the yellow node. However, if the application of the search method is modified and nodes are not marked as visited, search methods will perform circles, and they will not be able to find the solutions. Thus, direct application of the search methods is not feasible for

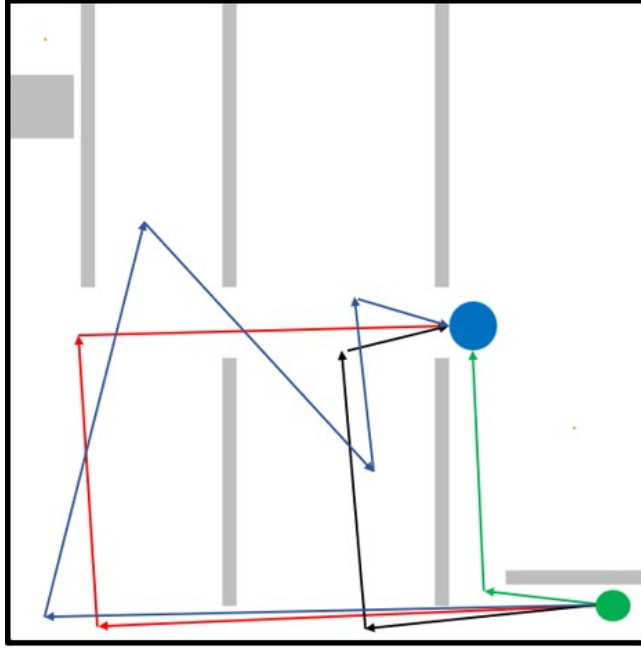


Figure A.1: Example Scenario

this problem.

Figure A.4 shows the application of RRT. In figure A.4a, A.4b and A.4c, RRT successfully found solutions. However, the solutions are not close to optimal. Due to the lack of optimality, paths consist of many states, which is unsuitable for our problem since the number of drones is limited. Moreover, RRT only finds one path every time it is initialized and does not provide alternative paths to the target. Lastly, in figure A.4d, RRT could not find a solution before the termination condition, reaching 10,000 nodes.

Figure A.5 shows the application of RRT\*. In figure A.5 RRT\* successfully found solutions, and they are better in terms of optimality, based on the total length of the path, than RRT. However, the number of states in the paths is high, and paths are unsuitable for the number of drone constraints. Thus, founded paths need post-processing to reduce the required number of drones. Moreover, like RRT, RRT\* cannot find alternative paths.

Figure A.6 shows the application of RRT and RRT\* with a narrower passage. Both methods could not find a solution before the termination condition, reaching 10,000

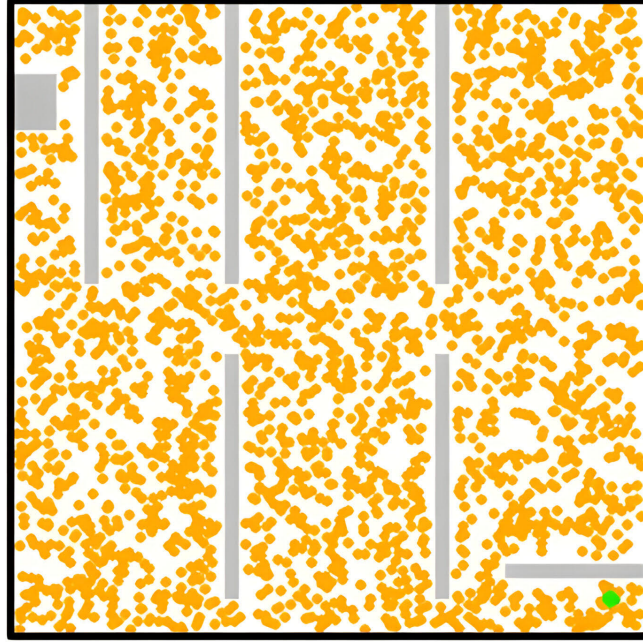


Figure A.2: Dense Sampled State-Space with 3000 nodes

nodes. The main reason for these results is that thin and thick obstacles are very close to each other, and any sample taken next to the goal node, the red node, is drawn to the right side of the thin obstacles since the nearest neighbors of the sample are at the right side.

Probabilistic Roadmap, PRM, has the advantage of being a multi-query method; however, the built roadmap does not consist of optimal paths. Direct application of a search method over the roadmap also suffers from the same problems and could not find alternative solutions mentioned for search methods. Like RRT and RRT\* methods, paths found at PRM also need post-process since the number of line segments is high.

PRM\* has the advantage of having optimal paths over PRM. However, just like PRM, direct application of a search method over the PRM\* roadmap cannot find alternative solutions. Like previous methods, paths found at PRM\* also need post-process since the number of line segments is high, although it is relatively less than previous methods.

Figure A.7 shows the application of the Visibility Roadmap for three different times.

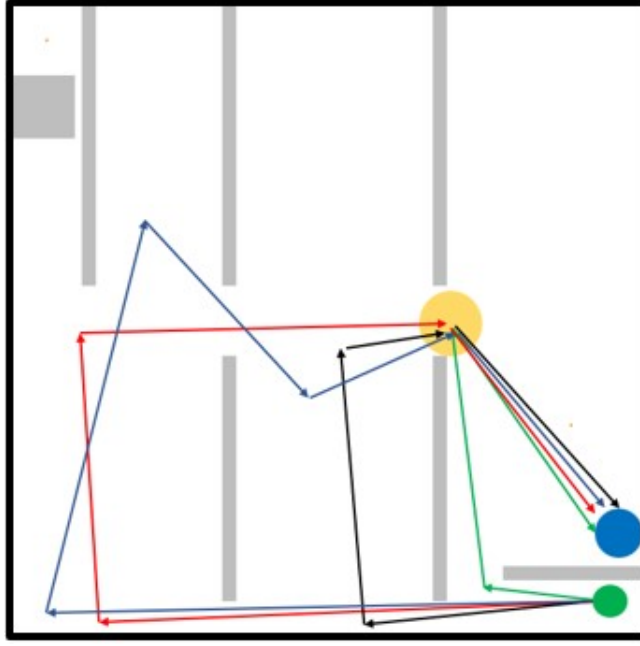
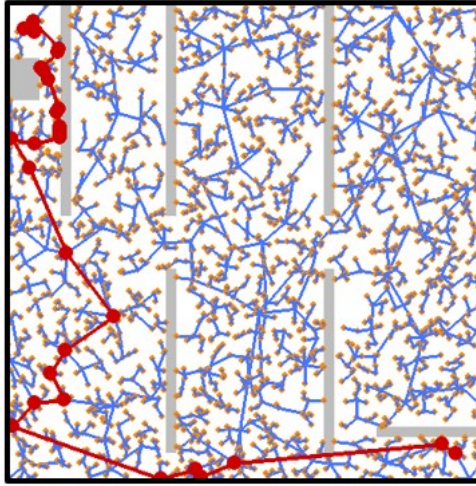


Figure A.3: Example scenario where multiple paths share the same node

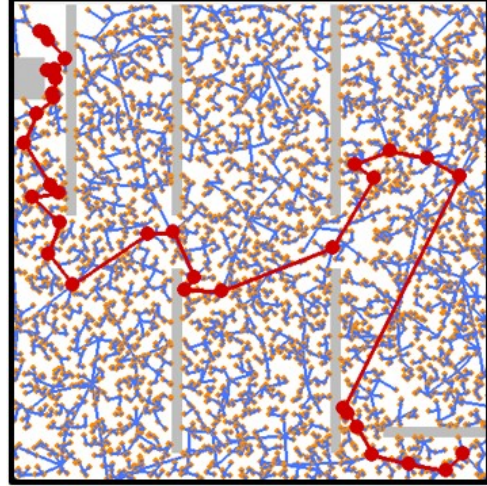
All of the roadmaps capture the explore the state-space completely. They consist of much fewer nodes compared to other roadmap methods. However, the roadmap does not consist of alternative paths, and paths are not even close to being optimal.

### A.1.2 Discrete Optimization Algorithms' Evaluation

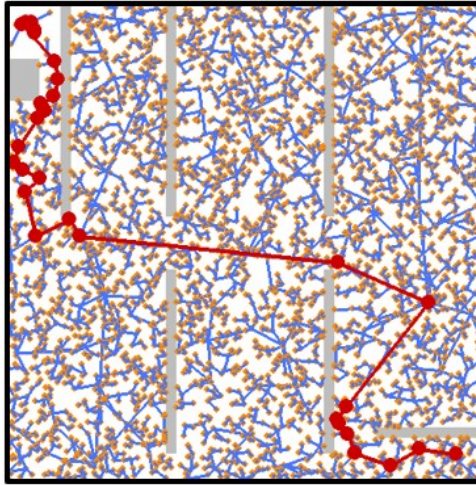
Unlike algorithms mentioned in section A.1.1, every discrete optimization algorithm mentioned in chapter 2 can be used in our problem. A common requirement for all discrete optimization algorithms is possible paths, which the planning algorithm should find. Even though all discrete optimization algorithms can be used in our problem, performance differences between algorithms are expected. Large Neighborhood Search, LNS, algorithm is chosen in line with the literature research since LNS outperforms other discrete optimization methods in scheduling and assignment problems in recent years. Moreover, LNS can better adapt to the appearances of the new targets since it optimizes sub-parts of the problems. Adding the new target requires optimizing only a sub-problem for LNS.



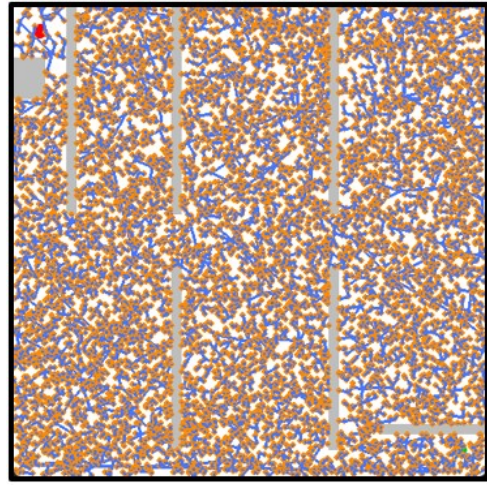
(a) 2549 nodes



(b) 4208 nodes

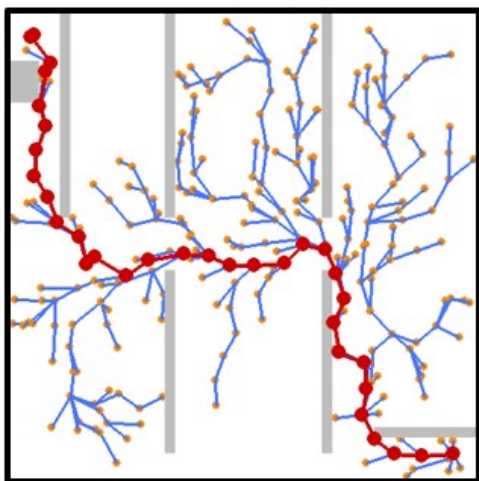


(c) 4284 nodes

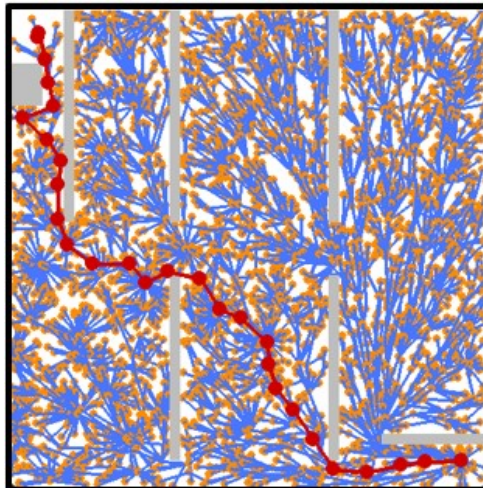


(d) 10,000 nodes

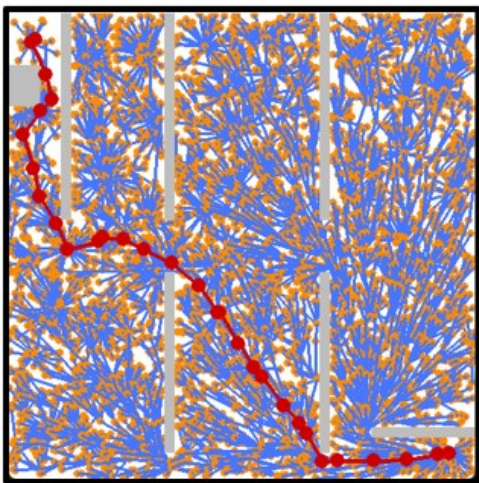
Figure A.4: Application of RRT



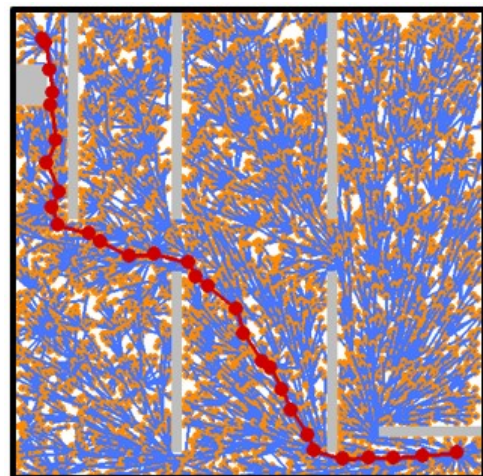
(a) 267 nodes



(b) 3470 nodes



(c) 4576 nodes



(d) 5816 nodes

Figure A.5: Application of RRT\*

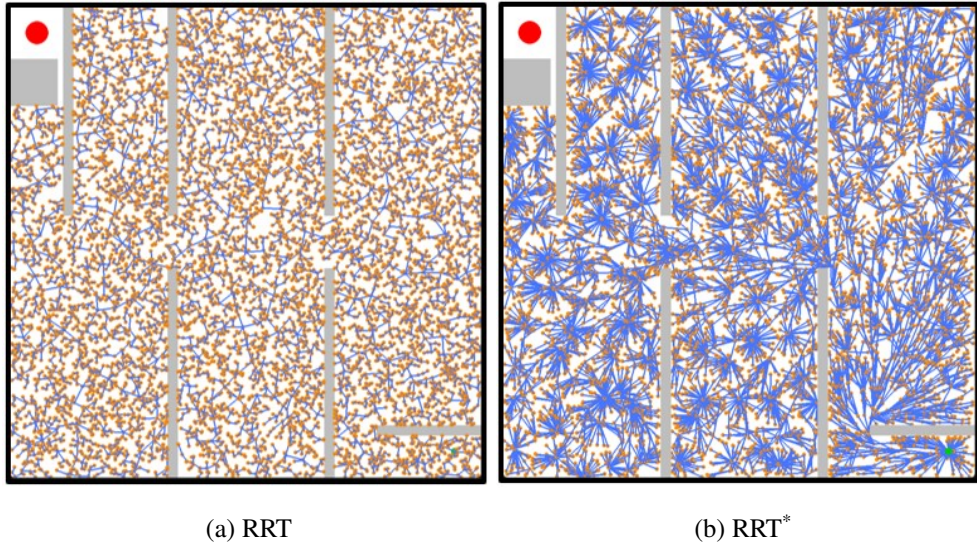


Figure A.6: Application of RRT and RRT\* with a narrower passage

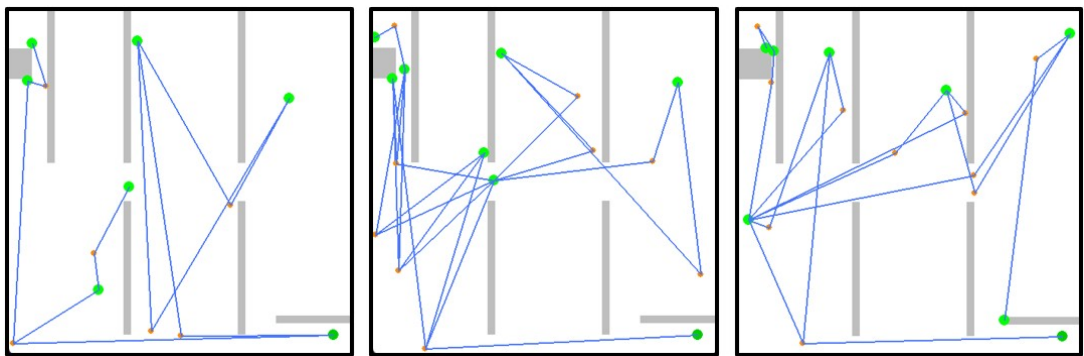


Figure A.7: 3 different Visibility Roadmaps over the state-space